TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Sistema de tutorización inteligente para la enseñanza de programación a niños

Diseño e implementación de una plataforma web con un sistema de tutorización inteligente para facilitar la enseñanza

Autor
Noelia Carrasco Vilar

Director
José Manuel Benítez Sánchez

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Noviembre de 2023

Sistema de tutorización inteligente para la enseñanza de programación a niños

Noelia Carrasco Vilar

Palabras clave: *Sistema de Tutorización Inteligente*, *Educación Infantil*, *Análisis*, *Automatización*, *Programación*

Resumen

La enseñanza de la programación a edades tempranas se ha vuelto fundamental en la sociedad actual. Es en este contexto que los Sistemas de Tutorización Inteligente (ITS) emergen como herramientas potenciales para apoyar un aprendizaje más personalizado y adaptado a las necesidades individuales de cada estudiante.

Este trabajo presenta el diseño, desarrollo y evaluación de un ITS enfocado en la enseñanza de programación. El sistema propuesto aplica una automatización en la corrección de los ejercicios con tal de poder ofrecer una buena retroalimentación del entendimiento del estudiante en tiempo real. Asimismo, es capaz de adaptarse a las necesidades individuales de los estudiantes y monitorizar su progreso.

El proceso de desarrollo del ITS se llevó a cabo en diversas fases. En primer lugar, se establecieron los fundamentos pedagógicos y se identificaron las necesidades específicas del público infantil. Posteriormente, se integraron técnicas avanzadas de corrección y elección de ejercicios con tal de garantizar la adaptabilidad y efectividad del sistema.

Para evaluar la eficacia, se realizó una prueba con un pequeño grupo de estudiantes, en colaboración con la empresa CODELEARN S.L. Los resultados iniciales sugieren una posible mejora significativa en la retención de conocimientos y en el rendimiento académico de los participantes, validando así la propuesta de este TFG.

En conclusión, este Trabajo de Fin de Grado aporta una solución innovadora al campo de la educación tecnológica infantil, demostrando la viabilidad y eficacia de los Sistemas de Tutorización Inteligente dentro del

mundo educativo de la programación.

Ïntelligent Tutoring System for Teaching Programming to Children

Noelia Carrasco Vilar

Keywords: *Intelligent Tutoring System*, *Early Childhood Education*, *Analysis*, *Automation*, *Programming*

Abstract

Teaching programming at an early age has become essential in today's society. It is in this context that Intelligent Tutoring Systems (ITS) emerge as potential tools to support more personalized learning tailored to the individual needs of each student.

This work presents the design, development, and evaluation of an ITS focused on teaching programming to children. The proposed system applies automation techniques in exercise correction to provide real-time feedback on the student's understanding. It is also capable of adapting to individual student needs and monitoring their progress.

The development of the ITS was carried out in various phases. Firstly, pedagogical foundations were established, and the specific needs of the child audience were identified. Advanced correction and exercise selection techniques were then integrated to ensure the adaptability and effectiveness of the system.

To evaluate the effectiveness of the ITS, tests and experiments were conducted with a small group , in collaboration with the company CODELEARN S.L. Initial results suggest a possible significant improvement in knowledge retention and academic performance of the participants, thus validating the proposal of this Bachelor's Thesis.

In conclusion, this Bachelor's Thesis provides an innovative solution to the field of early childhood technological education, demonstrating the feasibility and effectiveness of Intelligent Tutoring Systems in the educational programming world.

Yo, **Noelia Carrasco Vilar**, alumna de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 39475493H, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Noelia Carrasco Vilar

Granada a 13 de Noviembre de 2023.

D. **José Manuel Benítez Sánchez**, catédratico del departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Sistema de tutorización inteligente para la enseñanza de programación a niños***, ha sido realizado bajo su supervisión por **Noelia Carrasco Vilar**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 13 de Noviembre de 2023.

El director:

José Manuel Benítez Sánchezz

Agradecimientos

Quiero comenzar agradeciendo a loz profesores de la carrera que han realizado un compromiso para la formación académica de mi promoción. Su pasión y dedicación fomentaron mi interés por la informática, lo cual me ha llevado a realizar este trabajo de fin de grado. Un agradecimiento especial a mi tutor del TFG, quien además de guiar este proyecto, avivó mi interés por la investigación en los Sistemas de Tutorización Inteligente.

Agradezco de manera especial a la empresa Codelearn S.L. por su apoyo y colaboración en este proyecto. Su experiencia y recursos han sido cruciales para el desarrollo de mi TFG.

No puedo pasar por alto el papel fundamental que mi familia ha desempeñado en mi vida, tanto académica como personal. A mis padres y a mi hermano, cuya influencia ha sido determinante en mi formación, dedico este trabajo y los proyectos futuros, como reconocimiento a todos los esfuerzos que han realizado para permitirme alcanzar mis metas académicas. También deseo honrar la memoria de mis abuelos, quienes, presentes o ausentes, han sido una fuente de inspiración constante a lo largo de mi trayectoria.

Índice general

1. Introducción **1**
1.1. Contexto Actual y Relevancia de la Programación 1
1.2. Descripción del Problema 1
1.3. Enfoque 2
1.3.1. Pilares del Proyecto 2
1.3.2. Desarrollo Ágil 2
1.4. Colaboración con Codelearn S.L. 2
1.5. Conclusión 3

2. Estado del Arte y Trabajos Relacionados **5**
2.1. Definición de los ITS 5
2.2. Evolución de los ITS 5
2.3. Aplicaciones actuales de los ITS 6
2.3.1. Educación K-12 7
2.3.2. Educación Superior 7
2.3.3. Formación y Capacitación Corporativa 7
2.3.4. Salud y Medicina 7
2.3.5. Defensa y Aviación 8
2.3.6. Juegos Educativos y Simuladores 8
2.4. Limitaciones y Desafíos 8

3. Planificación **11**
3.1. Fases 11
3.1.1. Fase Inicial 11
3.1.2. Fase de análisis 11
3.1.3. Fase de desarrollo 12
3.1.4. Fase de prueba y corrección 13
3.1.5. Elaboración de la memoria 13
3.2. Presupuesto 14
3.2.1. Licencias 14
3.2.2. Recursos Materiales 15
3.2.3. Costes de Personal 15
3.2.4. Otros costes 15

3.2.5. Total 16

4. Requisitos **17**
4.1. Requisitos Funcionales 17
4.1.1. Generales 18
4.1.2. Sistema 19
4.1.3. Estudiantes 20
4.1.4. Profesores 22
4.1.5. Administrador 24
4.2. Requisitos No Funcionales 26

5. Análisis **29**
5.1. Análisis 29
5.1.1. Análisis de Usuarios Objetivo 29
5.1.2. Análisis de Requisitos Educativos 30
5.2. Modelos de Caso de Uso y Escenarios de Uso 30
5.2.1. Actores del Sistema 30
5.2.2. Restricciones y Reglas de Negocio 32
5.2.3. Escenarios de Uso Detallados 32
5.2.4. Seguridad y Acceso 35
5.3. Modelado de Entidades y sus Relaciones 36

6. Diseño e Implementación **37**
6.1. Diseño Pedagógico 37
6.2. Arquitectura del Sistema 38
6.3. Diseño de Base de Datos 39
6.4. Interfaz de Usuario 41
6.4.1. *Mockups* de la página web 41
6.4.2. Interacción Usuario-Sistema 46
6.5. Implementación de Algoritmos 47
6.5.1. Corrección y evaluación de calidad del código 47
6.5.2. Selección y evaluación de ejercicios 48
6.5.3. Cálculo de la puntuación 49
6.5.4. Comprobaciones del código 50
6.5.5. Cálculo del número extra de ejercicios 52
6.6. Implementación de Módulos de Aprendizaje Adaptativo . . . 54

7. Pruebas y Resultados **57**

8. Conclusiones y Líneas de Trabajo Futuro **59**

Anexos **60**

A. Uso de la Plataforma **61**

Glosario **62**

Bibliografía **63**

Bibliografía **65**

Índice de figuras

3.1. Diagrama de Gantt con la planificación del TFG 14

5.1. Diagrama de Caso de Uso del sistema 31
5.2. Entidad Relación del sistema 36

6.1. Arquitectura de sistema 38
6.2. Diagrama del modelo de datos 40
6.3. Inicio de Sesión para cualquier usuario. 41
6.4. Registro para cualquier usuario. 41
6.5. Página Principal de la interfaz del estudiante. 42
6.6. Apartado de Juegos de la interfaz del alumno. 42
6.7. Visualización de la teoría para el estudiante. 42
6.8. Apartado de contacto con el profesor. 43
6.9. Apartado de Ranking de la interfaz del usuario. 43
6.10. Visualización de los ejercicios para el usuario. 43
6.11. Apartado donde los estudiantes pueden revisar conceptos teóricos. 44
6.12. Apartado de configuración del estudiante. 44
6.13. Dashboard principal del profesor. 44
6.14. Pantalla de Estadísticas del profesor. 45
6.15. Pantalla con la lista de corrección de ejercicios. 45
6.16. Panel para facilitar la corrección los ejercicios para el profesor. 45
6.17. Apartado de Configuración del profesor. 46
6.18. Visión general del panel administrativo. 46
6.19. Diagrama de flujo para la selección de ejercicios 48
6.20. Diagrama de flujo para la selección de ejercicios 49
6.21. Diagrama de flujo que ilustra cómo se calcula la puntuación . 50
6.22. Diagrama de flujo que ilustra cómo se calcula la puntuación . 51
6.23. Diagrama de flujo que ilustra el cálculo del número extra de ejercicios 53

Índice de tablas

3.1. Costes asociados al personal para un ingeniero informático. . 15
3.2. Total de costes del proyecto 16

4.1. Requisito funcional RF-01: Inicio de sesión. 18
4.2. Requisito funcional RF-02: Registro de usuario. 18
4.3. Requisito funcional RF-03: Perfil de usuario. 18
4.4. Requisito funcional RF-04: Selección de ejercicios por requisitos. 19
4.5. Requisito funcional RF-05: Corrección de ejercicios. 19
4.6. Requisito funcional RF-06: Compilador integrado. 19
4.7. Requisito funcional RF-07: Categorización de la teoría. 20
4.8. Requisito funcional RF-08: Repetición de ejercicios similares tras fallo. 20
4.9. Requisito funcional RF-09: Límite de tiempo para completar un ejercicio. 20
4.10. Requisito funcional RF-10: Resolución de dudas. 20
4.11. Requisito funcional RF-11: Consulta de los rankings. 21
4.12. Requisito funcional RF-12: Juegos en la web. 21
4.13. Requisito funcional RF-13: Consulta de los ejercicios corregidos. 21
4.14. Requisito funcional RF-14: Sistema de notificación. 21
4.15. Requisito funcional RF-15: Feedback sobre la solución. 22
4.16. Requisito funcional RF-16: Sistema de puntos. 22
4.17. Requisito funcional RF-17: Lista de usuarios. 22
4.18. Requisito funcional RF-18: Consulta de ejercicios realizados. . 22
4.19. Requisito funcional RF-19: Panel de resolución de preguntas. 23
4.20. Requisito funcional RF-20: Lista de estudiantes inactivos. . . 23
4.21. Requisito funcional RF-21: Tiempo de resolución de ejercicios. 23
4.22. Requisito funcional RF-22: Corrección de ejercicios. 23
4.23. Requisito funcional RF-23: Usuarios con múltiples fallos. . . . 24
4.24. Requisito funcional RF-24: Ejercicios realizados por día. . . . 24
4.25. Requisito funcional RF-25: Usuarios con alta tasa de errores. 24
4.26. Requisito funcional RF-26: Alta de profesor por administrador. 24
4.27. Requisito funcional RF-27: Añadir elementos. 25
4.28. Requisito funcional RF-28: Consultar elementos. 25

4.29. Requisito funcional RF-29: Editar elementos. 25
4.30. Requisito funcional RF-30: Eliminar elementos. 25
4.31. Requisito funcional RF-31: Orden global. 26
4.32. Requisito no funcional RNF-01: Usabilidad. 26
4.33. Requisito no funcional RNF-02: Rendimiento. 26
4.34. Requisito no funcional RNF-03: Seguridad. 27
4.35. Requisito no funcional RNF-04: Compatibilidad. 27
4.36. Requisito no funcional RNF-05: Escalabilidad. 27
4.37. Requisito no funcional RNF-06: Disponibilidad. 27
4.38. Requisito no funcional RNF-07: Tiempo de Respuesta. 28
4.39. Requisito no funcional RNF-08: Mantenimiento. 28

5.1. EU-1: Realización de un ejercicio 33
5.2. EU-2: Tiempo superado en un ejercicio 34
5.3. EU-3: Corrección de ejercicios por parte del profesor 34
5.4. EU-4: Gestión del contenido didáctico 35
5.5. EU-5: Resolución de dudas 35

Capítulo 1

Introducción

1.1. Contexto Actual y Relevancia de la Programación

En la era actual, la digitalización está en todas partes. Afecta desde cómo nos comunicamos hasta como llevamos a cabo de tareas cotidianas. Siendo la tecnología la impulsora de este cambio. Dentro de este contexto, la programación ya no es una mera herramienta técnica. Si no, que se ha convertido en una habilidad esencial del siglo XXI. No solo es importante para aquellos que buscan una carrera tecnológica. También es importante para el público que desea entender y utilizar eficazmente las herramientas digitales. Sin embargo, enseñar programación a niños presenta desafíos únicos. Debido a la complejidad del contenido como a las necesidades pedagógicas específicas.

1.2. Descripción del Problema

Tal como hemos hablado en la sección anterior, enseñar programación a niños es un desafío que no podemos ignorar. Existen diversas plataformas educativas, pero muchas tienen bastantes limitaciones. Ofrecen ejercicios y evaluaciones estáticas, sin tener en cuenta una adaptación al ritmo o habilidades del estudiante. Además, la mayoría, únicamente se centran en la corrección del código. Por ello, ignoran aspectos como la eficiencia del mismo y el uso de buenas prácticas de programación. Este enfoque limitado no solo afecta a los estudiantes. También deja a profesores y administradores sin las herramientas necesarias para interactuar de una forma efectiva con el sistema. Es por eso que hay una necesidad clara e importante de una solución más completa y adaptativa para la enseñanza de programación a niños.

1.3. Enfoque

Este TFG adopta un enfoque multidisciplinario, combinando elementos de pedagogía, tecnología educativa y diseño de sistemas. La naturaleza del trabajo exige la interacción de varias disciplinas con tal de abordar de manera eficaz los desafíos presentados por la enseñanza de programación a niños.

1.3.1. Pilares del Proyecto

El trabajo se estructura alrededor de varios núcleos esenciales:

1. Análisis del papel actual de la programación en la sociedad digital.
2. Evaluación de los desafíos pedagógicos en la enseñanza de programación a niños.
3. Diseño y estructura del Sistema de Tutorización Inteligente (ITS).
4. Metodología de adaptación del ITS basada en las respuestas y progreso del estudiante.
5. Integración del ITS en una plataforma web, enfocándose en la experiencia del usuario.
6. Experimentación y análisis en contextos reales de enseñanza.

1.3.2. Desarrollo Ágil

Para abordar la dinámica cambiante y las necesidades de adaptabilidad del proyecto, se ha optado por usar un enfoque de desarrollo ágil para el desarrollo del software. Esto, favorece un proceso iterativo e incremental, permitiendo varios ajustes rápidos ante cualquier cambio. Además, de ofrecer la posibilidad de realizar entregas parciales que se pueden evaluar y ajustar en fases iniciales.

Esta estrategia de desarrollo asegura flexibilidad, fomenta la colaboración y se centra en el usuario final. Garantizando, así, que las soluciones implementadas se alinean con las necesidades del ámbito de la enseñanza y el aprendizaje.

1.4. Colaboración con Codelearn S.L.

Este proyecto no habría sido posible sin la colaboración estrecha con Codelearn S.L. [2], una destacada entidad internacional de la educación tecnológica para niños. A pesar de los avances y logros de Codelearn S.L., se identificó la necesidad de evolucionar y adaptar su sistema pedagógico para hacerlo más personalizado y efectivo. Esta colaboración ha permitido fusionar

la experiencia y conocimientos de ambas partes, con el objetivo de desarrollar un ITS que permita mejorar la forma en que se enseña programación a niños.

1.5. Conclusión

Este TFG busca ofrecer soluciones sólidas y bien argumentadas a los retos actuales de la enseñanza en la programación. Mediante un enfoque innovador y una aplicación práctica, se busca establecer un fundamento sobre el cual puedan construirse futuros avances e investigaciones dentro del campo de la tecnología educativa adaptativa.

Capítulo 2

Estado del Arte y Trabajos Relacionados

En este segundo capítulo tiene como objetivo presentar todo el conocimiento teórico adquirido en preparación para la implementación y adaptación de una ITS, así como el estado actual de esta tecnología.

Este estudio está basado principalmente en las contribuciones presentadas en las conferencias de la *International Conference on Intelligent Tutoring Systems* de los años 2016 [11], 2020 [12] y 2021 [13]. Estas ediciones han sido fuentes valiosas de información sobre las últimas tendencias, técnicas y avances en el campo de la tutoría inteligente. A lo largo de este capítulo, se explorará el papel de los Sistemas de Tutoría Inteligente en la educación moderna, las metodologías empleadas para adaptar el aprendizaje a las necesidades individuales de los estudiantes y las perspectivas actuales y futuras en la adaptación y personalización del aprendizaje en línea.

2.1. Definición de los ITS

2.2. Evolución de los ITS

La idea de ser capaces de utilizar la tecnología para personalizar y mejorar la experiencia de aprendizaje no es nueva. Sin embargo, con el crecimiento exponencial de las tecnologías de la información y la computación, esta idea ha ido experimentando transformaciones radicales, dando lugar a lo que se conoce hoy en día como Sistemas de Tutorización Inteligente. A continuación, se presenta una pequeña revisión de la evolución de los ITS, trazando su desarrollo desde sus inicios hasta su estado actual.

Los ITS nacieron en la década de 1970, bajo la idea de que la tecnología podía ser utilizada para simular la tutoría humana. Estos primeros sistemas, limitados por la capacidad computacional de la época, se centraban en unos conocimientos muy específicos, como podían ser la geometría o el álgebra.

Utilizaban una lógica básica y simple para ofrecer retroalimentación y una guía basada en las respuestas de los estudiantes. A pesar de su simplicidad, fueron representación de un gran paso hacia la individualización del aprendizaje.

Con el progreso en la investigación de la Inteligencia Artificial (IA) en la década de 1980, los ITS comenzaron a incorporar técnicas más avanzadas. Estos sistemas no solo eran capaces de evaluar las respuestas de los estudiantes, sino también de comprender parcialmente el proceso de pensamiento detrás de esas respuestas. Así, el feedback proporcionado se volvía más personalizado y adaptativo, reflejando mejor las necesidades individuales de cada alumno.

La siguiente década trajo consigo una mayor comprensión de la importancia de la interacción humano-computadora en los procesos educativos. Donde se desarrollaron las ITS con unas interfaces más intuitivas y amigables para el usuario. Estos sistemas comenzaron a ofrecer unos mejores entornos de aprendizaje, incorporando multimedia, simulaciones y otros recursos interactivos. La idea era crear un entorno donde el estudiante no solo recibiera retroalimentación, sino que también pudiera explorar y aprender de una manera más autónoma.

En el siglo XXI, la proliferación de *Big Data* y el avance en técnicas de aprendizaje profundo permitieron que los ITS alcanzaran un nuevo nivel de sofisticación. Con la capacidad de procesar grandes cantidades de datos, estos sistemas modernos pueden detectar patrones y tendencias en el comportamiento del estudiante que antes eran invisibles. Usando esta información para predecir las necesidades de aprendizaje futuras y adaptar el contenido en tiempo real, proporcionando una experiencia de aprendizaje aún más personalizada.

El potencial de los ITS sigue siendo muy amplio. Con la constante evolución de la tecnología y una comprensión cada vez más profunda de la pedagogía, es probable que veamos sistemas aún más avanzados en el futuro cercano. Incluso, se espera que la integración de realidades virtual y aumentada, junto con avances en IA, genere ITS que ofrezcan experiencias de aprendizaje inmersivas y altamente adaptativas.

2.3. Aplicaciones actuales de los ITS

La relevancia actual de los ITS se extiende a una variedad de dominios y, con la evolución tecnológica, su precisión y adaptabilidad han alcanzado niveles sin precedentes. Aprovechando técnicas de aprendizaje automático, estos sistemas pueden reconocer patrones en el comportamiento del usuario y adaptarse en función de ello. A su vez, mediante el análisis semántico, comprenden de manera profunda las respuestas de los estudiantes, garantizando una retroalimentación más precisa y relevante. Además, algunos ITS modernos están incorporando Realidad Virtual y Aumentada, ofreciendo entornos de aprendizaje inmersivos que permiten interacciones novedosas

y atractivas con el contenido. A continuación, exploraremos algunas de las aplicaciones más prominentes de estos sistemas en el contexto contemporáneo.

2.3.1. Educación K-12

Los ITS se han establecido como herramientas fundamentales para complementar el aprendizaje tradicional dentro del ámbito escolar, ya sea en la educación primaria o en la eduación secundaria. Sistemas como *Dreambox* [7] para matemáticas o *Fast Forword* [16] para la lectura, utilizan técnicas de tutorización inteligente para adaptar el contenido a las necesidades individuales de cada estudiante, permitiendo una educación más personalizada.

2.3.2. Educación Superior

Las universidades y colegios alrededor del mundo están empezando a integrar ITS en sus plataformas de aprendizaje en línea. Sobre todo, estas instituciones los han comenzado a usar para los cursos masivos en línea (MOOCs), donde el sistema se encarga de tutorizar a miles de estudiantes simultáneamente, sin una presencia física importante del profesor.

Por ejemplo, el sistema *edX* [26], iniciado por Harvard y el MIT, ha integrado algoritmos de tutorización inteligente para ofrecer retroalimentación personalizada a los estudiantes en sus cursos MOOCs con más inscritos. Asimismo, la Universidad de Stanford, a través de la plataforma *Coursera* [14], también ha experimentado con herramientas de tutorización para mejorar la retención y comprensión de los estudiantes adscritos.

2.3.3. Formación y Capacitación Corporativa

El mundo empresarial también ha reconocido el valor estos sistemas para la formación continua de sus empleados. Las grandes corporaciones han comenzado a implementarlos dentro de sus programas de formación, permitiendo la identificación y corrección de carencias de habilidades en tiempo real.

Grandes corporaciones como Google y Microsoft ya han comenzado a implementarlo en sus programas de formación interna. Por ejemplo, el *Google's IT Support Certificate* [10], un programa desarrollado para formar especialistas de soporte en TI, utiliza los ITS para guiar a los estudiantes a través de una serie de labores técnicas, identificando áreas de mejora en tiempo real.

2.3.4. Salud y Medicina

En el campo médico, también están desempeñando un papel crucial en la formación de futuros profesionales. Concretamente, *Touch Surgery Simulator* [18] permite a los médicos practicar procedimientos quirúrgicos en un entorno

virtual, sino que también proporciona retroalimentación detallada basada en el rendimiento y decisiones tomadas durante la simulación.

2.3.5. Defensa y Aviación

Es importante destacar que la aviación y la defensa han estado al frente en la adopción de simuladores con capacidades de tutorización. Un gran ejemplo es el *F-35 Lightning II Training System* [17]. Este ayuda a capacitar los pilotos de combate, proporcionando escenarios adaptativos basados en las habilidades y el progreso del piloto.

2.3.6. Juegos Educativos y Simuladores

Los juegos educativos, como *DragonBox* [6] que enseña matemáticas, o *Civilization EDU* [9] que instruye sobre historia y estrategia, también integran sistemas de tutorización inteligente para adaptar el contenido y los desafíos a las habilidades del jugador, asegurando una curva de aprendizaje apropiada.

2.4. Limitaciones y Desafíos

A pesar de las prometedoras capacidades y aplicaciones de los ITS en diversos contextos educativos y de formación, estos sistemas no están exentos de limitaciones y desafíos.

La interacción con estos sistemas no logra capturar la profundidad y empatía de un tutor humano, lo que puede generar desconexión o frustración por parte del estudiante. Estos sistemas, siendo fruto de una confluencia de disciplinas muy variadas, plantean retos en su diseño y actualización, especialmente cuando se busca mantener su relevancia en campos en constante evolución.

La adaptabilidad de estos sistemas también se encuentra en una línea delicada. Por ser lo suficientemente generales para diversos contextos. Asimismo, como específicos para brindar una retroalimentación precisa. Siendo este equilibrio esencial para garantizar su eficacia. Cabe destacar, que con la recopilación constante de datos del usuario, se debe tener especial delicadeza en cuanto a la privacidad y seguridad. Es imperativo que estos sistemas ofrezcan unas garantías robustas con tal de proteger la información del alumno.

Por otro lado, a pesar de su potencial democratizador en el acceso a la educación, los ITS corren el riesgo de convertirse en herramientas elitistas, accesibles solo para aquellos con los medios para obtenerlos. Por ello, es crucial que se trabaje en su accesibilidad y asequibilidad. Y si bien estos sistemas ofrecen métricas de rendimiento, medir su impacto real en el aprendizaje y en la transferencia práctica del conocimiento sigue siendo un gran reto.

En resumen, se podría decir que los ITS se ven como una herramienta revolucionaria, pero es fundamental enfrentar y superar los desafíos mencio-

nados para garantizar que su implementación sea efectiva y equitativa en todos los contextos educativos.

Capítulo 3

Planificación

La planificación es un aspecto importante en cualquier proyecto. Nos permite asegurar una correcta ejecución y el logro de los objetivos establecidos. Por ello, en este capítulo, abordaremos la organización y estructuración del proyecto para con tal de cumplir el objetivo principal. Además, un diagrama de Gantt se incluirá, al final de la sección 3.1, para ofrecer una visión detallada del tiempo asignado a cada tarea.

3.1. Fases

3.1.1. Fase Inicial

En la fase inicial se establecieron las bases y objetivos del proyecto. Se reconoció la necesidad de una plataforma educativa digital que enseñe programación asimismo como sus buenas prácticas. Se llevaron a cabo reuniones preliminares para esbozar las características deseadas y el alcance del proyecto, teniendo siempre en mente la experiencia de usuario y la eficiencia en la corrección y retroalimentación por parte del usuario.

3.1.2. Fase de análisis

Durante esta etapa se realizó en una exploración profunda y meticulosa de las necesidades y los requerimientos clave para diseñar una plataforma de aprendizaje eficaz. Se buscó conseguir un entorno que proporcionará una buena retroalimentación. Se puso énfasis en un enfoque adaptativo y personalizado, teniendo en cuenta la diversidad de ritmos y estilos de aprendizaje. Sé investigaron tanto las demandas funcionales como las tecnológicas, y se llevó a cabo un análisis de la plataforma Codelearn S.L. con tal de garantizar que las decisiones tomadas fueran adecuadas y eficientes.

Antes de adentrarnos en la construcción de la plataforma, fue esencial definir con precisión qué funcionalidades eran fundamentales. A través de

reuniones, lecturas e investigación se definieron las siguientes características principales para el sistema:

- Un sistema de corrección automatizada que evalúe tanto la solución como la calidad del código.
- Un sistema de retroalimentación que ofrezca comentarios constructivos.
- Un camino de ejercicios que se adapte dinámicamente al progreso y habilidades del estudiante.
- Un esquema de progreso visual que motive y guíe a los estudiantes a través de su aprendizaje.
- Una plataforma sencilla y visual, diseñada específicamente para niños, que les permita interactuar con el contenido de manera intuitiva.
- Un compilador en tiempo real que permite la ejecución y prueba de códigos en la plataforma.
- Un sistema de soporte y tutoría basado en un mecanismo de preguntas y respuestas para asistir a los estudiantes con dudas específicas.

Con los requisitos y objetivos ya estuvieron claros, se pasó a la elección de tecnologías que se iban a usar. La decisión fue tomada en base la experiencia, ya que de esta manera se conseguía rápidamente la eficiencia. Para el Frontend, se usó HTML, CSS y JavaScript, gracias a que ofrecían unas interfaces de usuario fluidas y dinámicas. Se usó Flask para el Backend, ya que se adapta fácilmente con diversas librerías de Python. Asimismo, para la gestión de la base de datos, se usó por PostgreSQL, ya que garantizaba una estructura robusta y escalable.

3.1.3. Fase de desarrollo

La fase de desarrollo se centró en implementar las características clave identificadas durante la fase anterior. Las principales características desarrolladas fueron:

1. **Corrección automatizada de los ejercicios**: Se desarrolló un sistema de corrección automatizada que no solo verifica la solución del ejercicio, sino también la calidad y eficiencia del código. Concretamente, analiza la limpieza del código, la utilización correcta de las estructuras de datos y si existen bucles repetitivos o redundantes. Garantizando que los estudiantes no solo resuelven el problema, sino que también adopten buenas prácticas de codificación.

2. **Retroalimentación Personaliza**: Se debe ofrecer un buen *feedback* para que los estudiantes entiendan sus errores. Por ello, se proporcionan comentarios detallados y las sugerencias necesarias. De esta manera, se fortalece la comprensión y se facilita el aprendizaje.

3. **Caminos de Ejercicios Aleatorio**: Para poder proporcionar una buena experiencia de aprendizaje, el sistema tiene un conjunto de ejercicios basados en requisitos específicos de forma aleatoria. De esta manera, se consigue que cada vez que el estudiante se enfrenta un requisito, se le desafía con un conjunto de ejercicios diferentes, asegurando que los haga por comprensión y no por repetición. Es importante destacar, que si un estudiante no realiza correctamente un ejercicio, el sistema le proporcionará otro diferente.

4. **Esquema de progreso individualizado**: Se realizó un tablero que muestra dónde se sitúa el estudiante y qué le espera a continuación. De esta manera, se consigue una motivación e incentiva el esfuerzo.

Cabe destacar que esta fase también implicó una serie de pruebas iterativas y correcciones, permitiendo ajustes y mejoras en tiempo real en función del feedback recibido y las observaciones realizadas.

3.1.4. Fase de prueba y corrección

Tras finalizar la anterior fase, tocó centrarse en el testeo del prototipo y sus respectivos ajustes. A pesar de haber diseñado la página web en base la investigación teórica, se debía validar su usabilidad.

Se eligió un grupo de 2-3 estudiantes, para que interactuaran con la plataforma. Pidiéndoles que hicieran varias tareas y que compartieran sus impresiones, dificultades y recomendaciones.

La retroalimentación de estos estudiantes fue clave. A partir de sus comentarios, se pudieron efectuar mejoras importantes en la plataforma. Se solucionaron fallos técnicos y cuestiones de usabilidad, además de optimizar la calidad de la experiencia educativa. Consiguiendo que la plataforma se adaptará a las necesidades y retos de los estudiantes, pudiendo asegurar una experiencia educativa de calidad.

3.1.5. Elaboración de la memoria

Durante las fases previas del proyecto, se llevó a cabo una recolección de datos e información pertinente. Al aproximarnos a las etapas finales del desarrollo, dicha información fue estructurada y articulada de manera coherente y sistemática, siendo la creación de la memoria.

El proyecto tuvo inicio a principios de julio, durando cinco meses hasta su finalización. Es importante subrayar que el período previo a julio, aunque

incluyó actividades de investigación y lectura, no es considerado crucial para la estructura del proyecto. Esto se debe a que, a pesar de haber realizado dichas tareas, no se le dedicó una cantidad de tiempo significante debido a otras responsabilidades. Por esta razón, se presenta el diagrama de Gantt que refleja el periodo de desarrollo del proyecto:

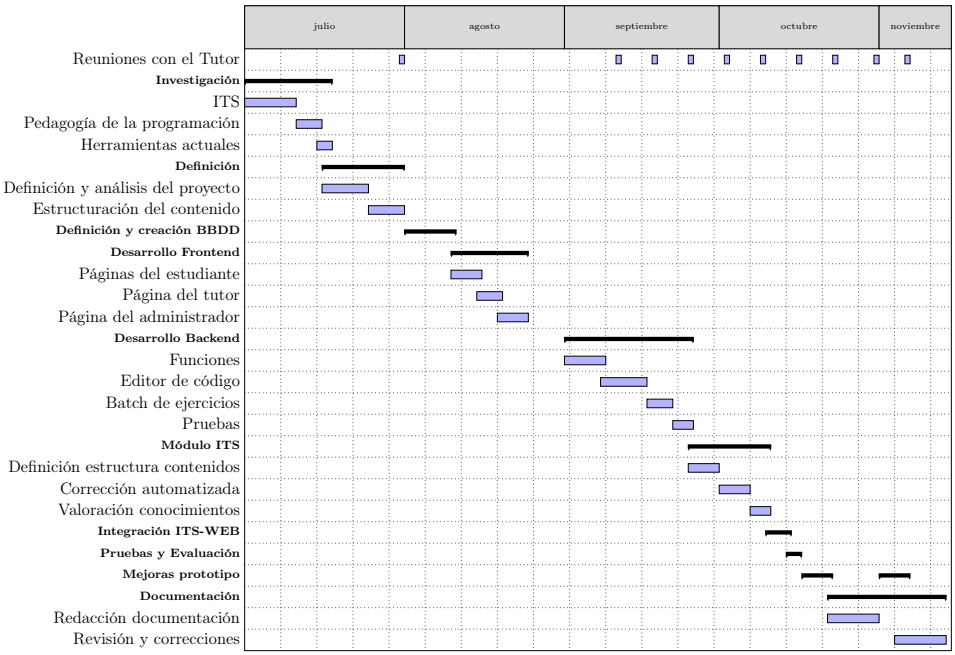

Figura 3.1: Diagrama de Gantt con la planificación del TFG

3.2. Presupuesto

En esta parte del documento se realiza una descripción de los gastos ocasionados por el proyecto.

3.2.1. Licencias

Con tal de realizar una buena evaluación del coste de un proyecto, se debe tener en cuenta todos aquellos gastos relacionados con licencias. En este caso, se optó por maximizar el uso de tecnologías *OpenSource* para reducir costos.

Para la interfaz de usuario, se usó HTML, CSS y JavaScript. En cuanto al Backend, se seleccionó Flask que también de costo cero. Además, se recurrió a PostgreSQL que no tiene ningún coste ni límite en cuanto a tamaño de la base de datos. Para el control de versiones, se usó Git y GitHub [24]. Aunque GitHub ofrece planes de pago, dispone de opciones gratuitas.

Por lo tanto, el coste de licencias es cero.

3.2.2. Recursos Materiales

Dentro de la inversión material, es indispensable considerar el equipo informático empleado en el proyecto, en este caso, un ordenador portátil. Comúnmente, estos dispositivos tienen una vida útil estimada que oscila entre los 2 y 4 años.

Si asumimos una vida útil intermedia de 3 años para el dispositivo, se estimará una depreciación anual del equipo. Por ello, considerando que el portátil tuvo un precio inicial de 950€, su depreciación anual será de 316,67€.

Finalmente, tenemos que tener en cuenta que el proyecto se ha desarrollado en una duración aproximada de 5 meses, por ello, el coste asociado al desgaste del portátil durante dicho período es de 132,78€.

3.2.3. Costes de Personal

Para el cálculo de costes de personal se ha tenido en cuenta un único empleado: un ingeniero informático. Tomando datos obtenidos de la Seguridad Social para el año 2023 [22], el salario de dicho ingeniero oscila entre una base mínima de cotización de 1.759,50€ y una base máxima de 4.495,50€ mensuales.

Concepto	**Coste Mensual (€)**	**Coste 5 Meses (€)**
Base Mínima de Cotización	1.759,50	8.797,50
Cotización Empresa (Base Mínima)	415,44	2.077,20
Cotización Trabajador (Base Mínima)	82,70	413,50
Total (Base Mínima)	**2.257,64**	**11.288,20**
Base Máxima de Cotización	4.495,50	22.477,50
Cotización Empresa (Base Máxima)	1.060,94	5.304,70
Cotización Trabajador (Base Máxima)	211,29	1.056,45
Total (Base Máxima)	**5.767,73**	**28.838,65**

Tabla 3.1: Costes asociados al personal para un ingeniero informático.

Si calculamos el promedio salarial, se obtendría aproximadamente unos 3.127,50€. Como el proyecto tiene una duración de 5 meses, el coste total en salarios sería de 15.637,50€ para el desarrollo completo del proyecto.

Adicionalmente, se debería considerar las cotizaciones correspondientes a la empresa y al trabajador, siendo el coste total para el proyecto de unos 17.900€ aproximadamente.

3.2.4. Otros costes

Este apartado hace alusión a todos aquellos gastos indirectos importantes para la realización del proyecto. Como, por ejemplo, el material de oficina, la conexión a Internet, la electricidad, los gastos de transporte, entre otros.

Se suele estimar este tipo de costes indirectos como un porcentaje del gasto principal, en este caso, del coste de personal. Siguiendo esta norma, se ha determinado que estos gastos representan aproximadamente el 10 % del gasto total en personal:

$$17.900 \times 0,10 = 1.790$$

Por lo tanto, se estima un coste adicional de 1.790€ para cubrir lo gastos indirectos asociados al proyecto.

3.2.5. Total

Para finalizar, se muestra una tabla 3.2 que resume todos los costes obtenidos, haciendo una estimación mensual y total del proyecto.

Concepto	**Coste Mensual (€)**	**Coste Total (5 Meses) (€)**
Licencias	0,00	0,00
Recursos Materiales (Portátil)	26,56	132,78
Personal (Salario promedio)	3.127,50	15.637,50
Cotizaciones (aprox.)	452,50	2.262,50
Otros costes	358,00	1.790,00
Total	**3964,06**	**19.822,78**

Tabla 3.2: Total de costes del proyecto

Capítulo 4

Requisitos

En este capítulo, se profundizará en el análisis de requisitos de la plataforma. Se tendrá en cuenta tanto los requisitos funcionales como los no funcionales, siendo este análisis de requisitos uno de los fundamentos para el desarrollo posterior de la plataforma.

4.1. Requisitos Funcionales

Con tal de garantizar la funcionalidad del programa y la realización de los objetivos establecidos, es esencial hacer una buena definición de los requisitos funcionales. Para facilitar su comprensión, presentaremos estos requisitos en tablas, siguiendo las convenciones de la ingeniería de requisitos y desarrollo de software. La organización de los contenidos es la siguiente:

- **ID**: Identificador único para cada requisito. Esencial para una fácil referencia y seguimiento.
- **Nombre**: Nombre del requisito.
- **Descripción**: Explicación del propósito y alcance del requisito.
- **Prioridad**: La importancia del requisito en relación con el sistema. Las categorías posibles son alta, media o baja. Siendo crucial abordar primero los requisitos de alta prioridad, mientras que los de baja prioridad se harán al final.
- **Estado**: Refleja la situación actual del requisito, pudiendo estar *en desarrollo*, *completado*, *en revisión* o *rechazado*.

4.1.1. Generales

ID	RF-01
Nombre	Inicio de sesión
Descripción	Los usuarios registrados deben poder acceder a la plataforma introduciendo su nombre de usuario y contraseña. El sistema debe garantizar la confidencialidad y autenticidad de esta operación, asegurando que las contraseñas se gestionan y almacenan de manera segura.
Prioridad	Alta
Estado	Completado

Tabla 4.1: Requisito funcional RF-01: Inicio de sesión.

ID	RF-02
Nombre	Registro de usuario
Descripción	Los nuevos usuarios deben poder registrarse en la plataforma proporcionando información relevante.
Prioridad	Alta
Estado	Completado

Tabla 4.2: Requisito funcional RF-02: Registro de usuario.

ID	RF-03
Nombre	Perfil de usuario
Descripción	Cada usuario debe tener un perfil donde pueda visualizar su información personal, y donde poder cambiar ciertos datos.
Prioridad	Media
Estado	Completado

Tabla 4.3: Requisito funcional RF-03: Perfil de usuario.

4.1.2. Sistema

ID	RF-04
Nombre	Selección de ejercicios por requisitos
Descripción	El sistema deberá presentar a los estudiantes ejercicios basados en el requisito que estén estudiando. Una vez completado un ejercicio, el siguiente deberá ser mostrado de forma aleatoria dentro del mismo requisito, hasta que se considere que el estudiante ha asimilado ese requisito.
Prioridad	Alta
Estado	Completado

Tabla 4.4: Requisito funcional RF-04: Selección de ejercicios por requisitos.

ID	RF-05
Nombre	Corrección de ejercicios
Descripción	Al enviar una solución, el sistema no solo verificará si es correcta, sino que también evaluará la calidad del código en términos de optimización y buenas prácticas de programación.
Prioridad	Alta
Estado	Completado

Tabla 4.5: Requisito funcional RF-05: Corrección de ejercicios.

ID	RF-06
Nombre	Compilador integrado
Descripción	La plataforma deberá contener con un compilador integrado que permita a los estudiantes probar sus códigos en Java, HTML+CSS+JS, Python y C++.
Prioridad	Alta
Estado	Completado

Tabla 4.6: Requisito funcional RF-06: Compilador integrado.

ID	RF-07
Nombre	Categorización de la teoría
Descripción	La teoría se clasificará en función del requisito, facilitando así su visualización dependiendo del punto en que se encuentre el estudiante.
Prioridad	Media
Estado	Completado

Tabla 4.7: Requisito funcional RF-07: Categorización de la teoría.

ID	RF-08
Nombre	Repetición de ejercicios similares tras fallo
Descripción	Si un estudiante falla un ejercicio, el sistema deberá presentar otro ejercicio con características similares para que el estudiante pueda intentarlo de nuevo.
Prioridad	Alta
Estado	Completado

Tabla 4.8: Requisito funcional RF-08: Repetición de ejercicios similares tras fallo.

ID	RF-09
Nombre	Límite de tiempo para completar un ejercicio
Descripción	Si el estudiante tarda más de un tiempo predefinido en completar un ejercicio, el sistema lo marcará como erróneo y presentará un nuevo ejercicio.
Prioridad	Media
Estado	Completado

Tabla 4.9: Requisito funcional RF-09: Límite de tiempo para completar un ejercicio.

4.1.3. Estudiantes

ID	RF-10
Nombre	Resolución de dudas
Descripción	Los estudiantes realizarán preguntas y adjuntar archivos para que el profesor les ayude.
Prioridad	Alta
Estado	Completado

Tabla 4.10: Requisito funcional RF-10: Resolución de dudas.

ID	RF-11
Nombre	Consulta de los rankings
Descripción	Los estudiantes podrán consultar un ranking donde vean los *top* 5 estudiantes diarios, semanales y mensuales.
Prioridad	Media
Estado	Completado

Tabla 4.11: Requisito funcional RF-11: Consulta de los rankings.

ID	RF-12
Nombre	Juegos en la web
Descripción	Por cada módulo que los estudiantes completen, se les desbloqueará un juego como recompensa.
Prioridad	Baja
Estado	Pendiente

Tabla 4.12: Requisito funcional RF-12: Juegos en la web.

ID	RF-13
Nombre	Consulta de los ejercicios corregidos
Descripción	Los estudiantes podrán consultar los comentarios del profesor sobre los ejercicios que haya corregido.
Prioridad	Media
Estado	Completado

Tabla 4.13: Requisito funcional RF-13: Consulta de los ejercicios corregidos.

ID	RF-14
Nombre	Sistema de notificación
Descripción	Se notificará al estudiante si su ejercicio es correcto o incorrecto cuando lo evalúe el profesor, junto con una retroalimentación del por qué es incorrecto.
Prioridad	Alta
Estado	Completado

Tabla 4.14: Requisito funcional RF-14: Sistema de notificación.

ID	RF-15
Nombre	Feedback sobre la solución
Descripción	Cuando un alumno envíe la solución, se le mostrará un feedback completo sobre ella.
Prioridad	Alta
Estado	Completado

Tabla 4.15: Requisito funcional RF-15: Feedback sobre la solución.

ID	RF-16
Nombre	Sistema de puntos
Descripción	Por cada ejercicio completado, el estudiante obtendrá una cantidad de puntos dependiendo de la calidad de su solución.
Prioridad	Baja
Estado	Completado

Tabla 4.16: Requisito funcional RF-16: Sistema de puntos.

4.1.4. Profesores

ID	RF-17
Nombre	Lista de usuarios
Descripción	El profesor podrá ver una lista de todos los usuarios registrados en la plataforma.
Prioridad	Media
Estado	Completado

Tabla 4.17: Requisito funcional RF-17: Lista de usuarios.

ID	RF-18
Nombre	Consulta de ejercicios realizados
Descripción	El profesor podrá consultar los ejercicios realizados por los estudiantes, junto con su estado,
Prioridad	Media
Estado	Completado

Tabla 4.18: Requisito funcional RF-18: Consulta de ejercicios realizados.

ID	RF-19
Nombre	Panel de resolución de preguntas
Descripción	El profesor tendrá un panel donde podrá responder las preguntas realizadas por los estudiantes.
Prioridad	Alta
Estado	Completado

Tabla 4.19: Requisito funcional RF-19: Panel de resolución de preguntas.

ID	RF-20
Nombre	Lista de estudiantes inactivos
Descripción	El profesor podrá consultar en una lista los estudiantes que llevan más de una semana sin conectarse, incluyendo sus correos electrónicos para un posible seguimiento.
Prioridad	Media
Estado	Completado

Tabla 4.20: Requisito funcional RF-20: Lista de estudiantes inactivos.

ID	RF-21
Nombre	Tiempo de resolución de ejercicios
Descripción	Habrá una lista de estudiantes que tengan un tiempo de resolución de ejercicios demasiado bajo o alto, con tal de identificar posibles problemas o áreas de mejora.
Prioridad	Baja
Estado	Completado

Tabla 4.21: Requisito funcional RF-21: Tiempo de resolución de ejercicios.

ID	RF-22
Nombre	Corrección de ejercicios
Descripción	El profesor tendrá la capacidad de corregir los ejercicios, relacionados con HTML, enviados por los estudiantes y ser capaz de proporcionar una retroalimentación.
Prioridad	Alta
Estado	Completado

Tabla 4.22: Requisito funcional RF-22: Corrección de ejercicios.

ID	RF-23
Nombre	Usuarios con múltiples fallos
Descripción	Se listarán los usuarios que han fallado más de dos veces un ejercicio específico con tal de poder identificar áreas problemáticas.
Prioridad	Baja
Estado	Completado

Tabla 4.23: Requisito funcional RF-23: Usuarios con múltiples fallos.

ID	RF-24
Nombre	Ejercicios realizados por día
Descripción	Habrá un gráfico donde se podrá consultar el número de ejercicios realizados por día en la plataforma. Con tal de poder evaluar la actividad y el compromiso de los estudiantes.
Prioridad	Baja
Estado	Completado

Tabla 4.24: Requisito funcional RF-24: Ejercicios realizados por día.

ID	RF-25
Nombre	Usuarios con alta tasa de errores
Descripción	Habrá una lista de usuarios con una alta tasa de errores en los ejercicios, con tal de poder ofrecer asistencia adicional.
Prioridad	Baja
Estado	Completado

Tabla 4.25: Requisito funcional RF-25: Usuarios con alta tasa de errores.

4.1.5. Administrador

ID	RF-26
Nombre	Alta de profesor por administrador
Descripción	El administrador tendrá la capacidad de registrar a un profesor mediante el panel de administración.
Prioridad	Alta
Estado	Completado

Tabla 4.26: Requisito funcional RF-26: Alta de profesor por administrador.

ID	RF-27
Nombre	Añadir elementos
Descripción	El administrador podrá añadir nuevos módulos, requisitos, profesores, teoría y ejercicios a la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 4.27: Requisito funcional RF-27: Añadir elementos.

ID	RF-28
Nombre	Consultar elementos
Descripción	El administrador podrá consultar los detalles de los módulos, requisitos, profesores, teoría y ejercicios existentes en la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 4.28: Requisito funcional RF-28: Consultar elementos.

ID	RF-29
Nombre	Editar elementos
Descripción	El administrador podrá editar los detalles de los módulos, profesores, teoría y ejercicios, excepto los requisitos que no son editables.
Prioridad	Alta
Estado	Completado

Tabla 4.29: Requisito funcional RF-29: Editar elementos.

ID	RF-30
Nombre	Eliminar elementos
Descripción	El administrador podrá eliminar módulos, requisitos, profesores, teoría y ejercicios de la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 4.30: Requisito funcional RF-30: Eliminar elementos.

ID	RF-31
Nombre	Orden global
Descripción	El administrador podrá crear y consultar un orden global que especifica el camino que debe seguir el estudiante en la plataforma, siendo una secuencia de módulos y requisitos.
Prioridad	Alta
Estado	Completado

Tabla 4.31: Requisito funcional RF-31: Orden global.

4.2. Requisitos No Funcionales

Con tal de asegurar la calidad global del sistema, deberemos tener en cuenta los requisitos funcionales. Por ello, vamos a detallar estos requisitos de manera estructurada siguiendo el mismo formato que con los requisitos funcionales.

ID	RNF-01
Nombre	Usabilidad
Descripción	El sistema deberá ser intuitivo y fácil de usar para los diferentes tipos de usuarios.
Prioridad	Alta
Estado	Completado

Tabla 4.32: Requisito no funcional RNF-01: Usabilidad.

ID	RNF-02
Nombre	Rendimiento
Descripción	El sistema deberá ser capaz de manejar una cantidad promedio de usuarios.
Prioridad	Media
Estado	Completado

Tabla 4.33: Requisito no funcional RNF-02: Rendimiento.

ID	RNF-03
Nombre	Seguridad
Descripción	Todos los datos sensibles, como contraseñas, deben estar encriptados.
Prioridad	Alta
Estado	Completado

Tabla 4.34: Requisito no funcional RNF-03: Seguridad.

ID	RNF-04
Nombre	Compatibilidad
Descripción	El sistema debe ser compatible con los navegadores web más comunes, incluidos Chrome, Firefox y Safari.
Prioridad	Alta
Estado	Completado

Tabla 4.35: Requisito no funcional RNF-04: Compatibilidad.

ID	RNF-05
Nombre	Escalabilidad
Descripción	El sistema deberá ser escalable para permitir la adición de más módulos, ejercicios y usuarios en el futuro.
Prioridad	Alta
Estado	Completado

Tabla 4.36: Requisito no funcional RNF-05: Escalabilidad.

ID	RNF-06
Nombre	Disponibilidad
Descripción	La página web debe estar disponible el 99 % del tiempo, excluyendo el tiempo de mantenimiento programado.
Prioridad	Media
Estado	Completado

Tabla 4.37: Requisito no funcional RNF-06: Disponibilidad.

ID	RNF-07
Nombre	Tiempo de Respuesta
Descripción	El tiempo de carga de las páginas no debe superar los 5 segundos.
Prioridad	Media
Estado	Completado

Tabla 4.38: Requisito no funcional RNF-07: Tiempo de Respuesta.

ID	RNF-08
Nombre	Mantenimiento
Descripción	Se deberá tener una arquitectura bien estructurada que facilite poder añadir nuevas funcionalidades. Además, de poder corregir errores sin afectar al funcionamiento de todo el sistema
Prioridad	Alta
Estado	Completado

Tabla 4.39: Requisito no funcional RNF-08: Mantenimiento.

Capítulo 5

Análisis

5.1. Análisis

En este capítulo se van a exponer diferentes aspectos relacionados con el diseño e implementación. Concretamente, se realiza un análisis de los usuarios objetivo, comprendiendo sus necesidades, expectativas y limitaciones. Asimismo, también se muestran los modelos de caso de uso, la arquitectura del sistema. Conjuntamente, con los modelos de comportamiento y el modelado de entidades y relaciones. De esta manera, se ofrece una visión clara y estructurada de la solución propuesta.

5.1.1. Análisis de Usuarios Objetivo

El enfoque de la plataforma web está dirigido principalmente a niños entre los 8 y 18 años, siguiendo la metodología de CODELEARN S.L.. Donde la mayoría de los clientes son estudiantes con poco o ningún conocimiento previo de programación. Es por ello que la plataforma está diseñada para ser intuitiva y flexible, en línea con las recomendaciones de Large y Beheshti sobre el diseño de interfaces para niños [15].

Si tenemos en cuenta la brevedad del período de atención de los niños y su desarrollo cognitivo en curso, se deberán evitar tareas extensas o excesivamente complicadas. Las expectativas y necesidades son diversas, pero comúnmente incluyen una interfaz visualmente atractiva, instrucciones claras y una retroalimentación inmediata tal como especifican Duglas Frye y Elliot Soloway [8]. Asimismo, se han incorporado elementos de gamificación dentro la plataforma con tal mantener el interés y la motivación de los usuarios.

Finalmente, la plataforma será accesible; es decir, se podrá acceder desde múltiples dispositivos como ordenadores, tablets y smartphones debido a su diseño *Web Responsive*.

5.1.2. Análisis de Requisitos Educativos

La enseñanza de habilidades de programación dentro de las escuelas está cobrando una gran relevancia [23]. Sin embargo, hay una falta de herramientas y enfoques efectivos mucho más allá de herramientas como Scratch [23]. Conscientes de esta brecha, este sistema educativo adopta un enfoque fundamentado en evidencia y prácticas académicas sólidas.

Aunque lenguajes como Scratch y Logo son de los más populares para la educación de niños, se ha optado por Python, C++, Java y HTML+CSS+JS. Python es conocido por su simplicidad y legibilidad, lo que facilita el aprendizaje para los principiantes [19]. Además, permite una transición fluida a lenguajes más complejos. C++, Java y HTML+CSS+JS, aunque sujetos a debate sobre su idoneidad para principiantes, son cruciales en la industria y la educación. Su relevancia práctica justifica su presencia [5].

Al ir más allá de los enfoques basados en bloques como Scratch, se busca que los niños aprendan la programación basada en texto, más cercana a la realidad. Los ejercicios se organizan por competencias específicas, permitiendo un aprendizaje adaptable y progresivo. Alineando el enfoque con los resultados de investigaciones sobre el tema [21].

Es importante destacar que el sistema se ha diseñado para ser inclusivo, ofreciendo ejercicios de diversa dificultad y con un buen sistema de asistencia. De esta manera, se siguen las directrices recomendadas por Luxton-Reily y Wünsche sobre las características de un buen sistema inteligente para la enseñanza de programación [3], consiguiendo un buen enfoque educativo.

5.2. Modelos de Caso de Uso y Escenarios de Uso

En esta sección se va a hablar de las interacciones usuario-sistema. En primer lugar, se abordará los modelos de caso de uso 5.1. Debido a que nos proporcionan una visión panorámica, resaltando las interacciones posibles y los actores implicados. Es decir, nos ayudará a definir lo que el sistema puede hacer y lo que se espera que haga. Además, también se va a especificar los diferentes escenarios de uso, ya que nos proporcionan una mejor comprensión de las interacciones específicas. Pudiendo observar las verdaderas dinámicas del sistema en acción.

Con esta combinación, se establecen las funcionalidades esperadas del sistema y cómo este responderá ante diferentes situaciones y acciones del usuario. Cabe destacar que los diagramas se han realizado mediante PlantULM [1].

5.2.1. Actores del Sistema

La interacción de la plataforma se basa en tres tipos de usuarios: estudiantes, profesores y administradores. Es importante destacar que el acceso a

la plataforma está restringido a usuarios registrados, ya que se busca asegurar que cada interacción dentro del sistema se realice en un entorno seguro y personalizado.

Estudiantes Los estudiantes son los principales usuarios. Su objetivo es aprender programación a través de los recursos educativos disponibles. Podrán acceder a ejercicios, leer material didáctico, recibir retroalimentación y consultar sus dudas.

Profesores Su papel es de facilitador de aprendizaje, y de apoyo emocional a los estudiantes. Corrigen los ejercicios de HTML, debido su complejidad de corrección. Además, de la resolución de dudas. También deben ir comprobando los estudiantes que más ejercicios fallen o estén mucho tiempo sin conectarse, para escribirles un correo con tal de motivarles.

Administradores Los administradores tienen el máximo nivel de control. Pueden agregar, consultar y editar todo el contenido didáctico de la plataforma. Además, de poder gestionar los profesores.

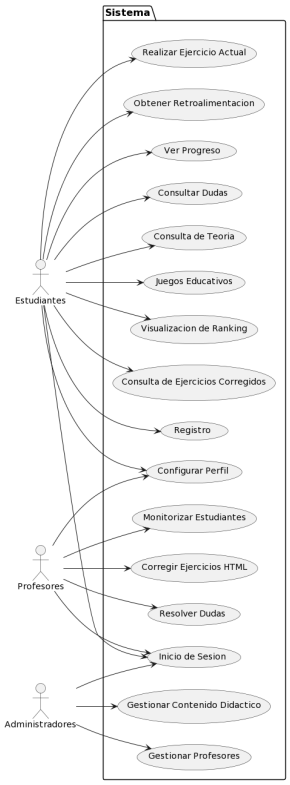

Figura 5.1: Diagrama de Caso de Uso del sistema

5.2.2. Restricciones y Reglas de Negocio

Las pautas operativas y limitaciones rigen la interacción de los usuarios con el sistema. Se abordarán las restricciones que afectan a distintos actores y se establecen las reglas que garantizan un entorno seguro y eficiente. Con tal de delinear las condiciones bajo las cuales se opera la plataforma educativa.

Restricciones de Acceso

- Estudiantes: Limitados a módulos y ejercicios asignados por la plataforma.
- Profesores: Acceso a corrección de ejercicios HTML y panel de dudas; sin permisos para modificar contenido.
- Administradores: Control total en la gestión de contenido y cuentas de profesores.

Reglas de Validación

- Los ejercicios tienen un tiempo límite para su realización; de excederlo, se considerarán fallidos.
- Puntuación basada en corrección y calidad del código.
- Validación manual por profesores para ejercicios HTML.

Reglas de Negocio Adicionales

- En caso de fallo en un ejercicio, el estudiante recibe un nuevo problema para evitar la desmotivación.
- Los profesores pueden identificar y enviar correos motivacionales a estudiantes con bajo rendimiento o actividad.
- Solo administradores pueden añadir o eliminar cuentas de profesores.

5.2.3. Escenarios de Uso Detallados

Dentro de esta subsección se ofrece un análisis de los escenarios de uso que involucran a los distintos tipos de usuarios. Concretamente, se va a presentar una serie de tablas que detallarán cada escenario de uso.

A continuación, se describen los elementos que compondrán cada tabla de escenarios de uso, siguiendo las pautas sugeridas por expertos en usabilidad:

- **ID**: Código de identificación único para cada escenario de uso.
- **Nombre**: Título descriptivo.

- **Actor**: Define los usuarios que están implicados en la acción.

- **Descripción**: Una breve explicación del escenario de uso.

- **Precondiciones**: Condiciones o eventos que deben cumplirse o suceder antes de que se inicie el escenario de uso.

- **Postcondición**: Resultados o estados que se alcanzan después de completar el escenario de uso.

- **Flujo**: Detalla todos los pasos implicados para la realización de cada caso de uso, incluyendo tanto el flujo básico como cualquier flujo alternativo.

Los escenarios están centrados en los factores más interesantes de la plataforma diseñada. Es importante destacar que se ha decidido omitir los casos de uso tales como inicio de sesión, registro o cerrar sesión con tal de evitar la redundancia debido a su lógica común.

ID	EU-1
Nombre	Realización de un ejercicio
Actor	Estudiante
Descripción	El estudiante resuelve un ejercicio de programación.
Precondiciones	Estudiante ha iniciado sesión y no ha completado todos los módulos.
Postcondición	El ejercicio se marca como completado y el estudiante recibe retroalimentación.
Flujo	1. Accede al módulo que aún no tenga completado. 2. Se le abre un ejercicio. 3. Resuelve el ejercicio dentro del tiempo límite. 4. Envía la solución. 5. Recibe retroalimentación y puntuación si es pertinente.

Tabla 5.1: EU-1: Realización de un ejercicio

ID	EU-2
Nombre	Tiempo superado en un ejercicio
Actor	Estudiante
Descripción	El estudiante no logra completar un ejercicio de programación dentro del tiempo límite.
Precondiciones	Estudiante ha iniciado sesión y está resolviendo un ejercicio.
Postcondición	El ejercicio se marca como erróneo y se le muestra un nuevo ejercicio.
Flujo	1. El estudiante está resolviendo un ejercicio. 2. El tiempo límite se alcanza antes de que el estudiante envíe la solución. 3. El ejercicio se marca como fallado. 4. Se le presenta un nuevo ejercicio al estudiante.

Tabla 5.2: EU-2: Tiempo superado en un ejercicio

ID	EU-3
Nombre	Corrección de ejercicios por parte del profesor
Actor	Profesor
Descripción	El profesor corrige manualmente ejercicios HTML debido a su complejidad.
Precondiciones	Profesor ha iniciado sesión, y algún estudiante ha enviado un ejercicio de HTML para corrección.
Postcondición	Ejercicio corregido y retroalimentación enviada al estudiante.
Flujo	1. Inicia sesión en la plataforma. 2. Accede a la lista de ejercicios HTML pendientes de corrección. 3. Selecciona un ejercicio para corregir. 4. Evalúa el ejercicio y escribe una retroalimentación. 5. Envía los datos y se marca el ejercicio como corregido.

Tabla 5.3: EU-3: Corrección de ejercicios por parte del profesor

ID	EU-4
Nombre	Administrador gestionando contenido didáctico
Actor	Administrador
Descripción	El administrador añade, edita o elimina contenido educativo en la plataforma.
Precondiciones	Administrador ha iniciado sesión.
Postcondición	Contenido educativo actualizado en la plataforma.
Flujo	1. Inicia sesión en la plataforma. 2. Accede al panel de administración de contenido didáctico. 3. Selecciona la opción para añadir, editar o eliminar según el tipo de contenido. 4. Realiza las acciones correspondientes. 5. Confirma los cambios.

Tabla 5.4: EU-4: Gestión del contenido didáctico

ID	EU-5
Nombre	Resolución de dudas
Actor	Estudiante, Profesor
Descripción	El estudiante plantea una duda y el profesor la resuelve.
Precondiciones	Estudiante y profesor han iniciado sesión.
Postcondición	Duda resuelta.
Flujo	1. Estudiante accede a la sección de dudas. 2. Escribe y envía su pregunta. 3. Profesor visualiza en el panel la duda. 4. Accede a la duda. 5. Resuelve la duda proporcionando una explicación detallada. 6. Estudiante recibe notificación de que su esta duda resuelta.

Tabla 5.5: EU-5: Resolución de dudas

5.2.4. Seguridad y Acceso

Para garantizar la integridad de la plataforma, es crucial abordar aspectos de seguridad y acceso.

Autenticación La plataforma adopta un sistema de autenticación. Esto se logra mediante el uso de correo electrónico y contraseña.

Autorización Los roles de usuario, es decir, estudiantes, profesores y administradores, vienen con diferentes niveles de acceso. Los profesores, por ejemplo, pueden corregir ejercicios y resolver dudas, pero no tienen permisos para alterar el contenido didáctico.

Seguridad de Datos La seguridad de los datos es una prioridad. Por ello, todas las transmisiones de datos se realizan mediante conexiones HTTPS seguras. Además, técnicas de hash y salting modernas aseguran que las contraseñas se almacenen de manera segura.

Monitorización y Auditoría Finalmente, mantenemos un registro de todas las actividades significativas en la plataforma. Esto permite a los administradores realizar un seguimiento detallado y, si es necesario, controlar las actividades de los usuarios.

5.3. Modelado de Entidades y sus Relaciones

El modelado de datos es fundamental para comprender la estructura y relaciones entre las diferentes entidades en una base de datos. En el contexto de este proyecto entidades como los estudiantes interactúan con elementos como módulos, ejercicios y teorías. Las relaciones entre estas entidades garantizan coherencia y eficiencia en el sistema. El Diagrama Entidad-Relación (ER) 5.2 proporciona una representación gráfica de estas interacciones.

Dentro del diagrama, se identifican entidades clave como Usuarios, Ejercicios, Teoría, Requisitos y Modulos. Las interacciones entre estas reflejan la lógica del negocio y las necesidades del sistema. Por ejemplo, un usuario podría estar trabajando en varios ejercicios al mismo tiempo, pero cada tarea realizada es única para ese usuario. De manera similar, un módulo puede contener varios ejercicios y temas teóricos, pero cada uno de estos elementos está vinculado exclusivamente a un módulo.

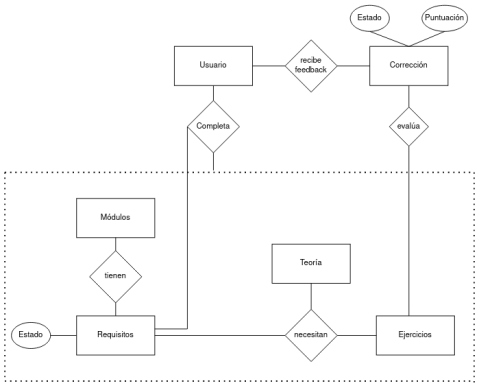

Figura 5.2: Entidad Relación del sistema

Capítulo 6

Diseño e Implementación

6.1. Diseño Pedagógico

El diseño pedagógico se centra en el aprendizaje basado en problemas [4]. Concretamente, se introduce la teoría relevante, que puede incluir explicaciones de texto, imágenes ilustrativas, y en ocasiones, material en formato de video. Una vez que el usuario ha absorbido el contenido teórico, se le presentan problemas prácticos relacionados para resolver.

Existen dos categorías principales de ejercicios, cada una con un peso distinto en la estructura de aprendizaje:

1. Ejercicios estándar: Son problemas prácticos que refuerzan la comprensión del usuario sobre el tema.

2. Ejercicios clave (*key_ exercise*): Estos ejercicios desempeñan un papel crucial en la progresión del usuario. Son indicativos de si el usuario ha comprendido los requisitos fundamentales del tema en cuestión. Se deben completar estos ejercicios para avanzar al siguiente tema o módulo. Si el usuario no logra superar estos ejercicios en un primer intento, se le presentan ejercicios adicionales para reforzar su comprensión antes de volver a intentarlo.

La plataforma corrige automáticamente para los módulos de C++, Java y Python, tal como se mencionó con anterioridad. Sin embargo, para el módulo de HTML+CSS+JS, se requiere una revisión manual por parte de un instructor. Esta revisión no solo califica el ejercicio, sino que también proporciona comentarios constructivos para guiar al usuario hacia la solución correcta.

Además de la corrección, la plataforma evalúa la limpieza y estructura del código presentado, ya que es esencial para un buen aprendizaje. Por lo tanto, se proporciona una retroalimentación detallada al estudiante no solo sobre la corrección de su respuesta, sino también sobre cómo mejorar y optimizar su código.

Finalmente, la plataforma tiene la capacidad de compilar ejercicios, permitiendo la verificación y corrección del código antes de su envío.

6.2. Arquitectura del Sistema

La interacción del usuario en el Frontend se lleva a cabo utilizando tecnologías estándar como HTML sin emplear *frameworks* adicionales para su desarrollo. Además, la interfaz es *Web Responsive*, lo que garantiza su adaptación a variados dispositivos. Desde el punto de vista de la seguridad, se han implementado medidas como la validación de entradas, previniendo así amenazas como los ataques XSS. Los datos de sesión se conservan en el cliente mediante cookies.

Por otro lado, el Backend se ha desarrollado con Flask. Se encarga de administrar las API RESTful y de establecer una conexión ininterrumpida con la base de datos PostgreSQL. La biblioteca *flask_ login* respalda las funcionalidades de autenticación y autorización, aportando una capa de seguridad adicional. La lógica de negocio del sistema reside principalmente en el Sistema de Tutoría Inteligente (ITS). Este subsistema es el encargado de la elección y evaluación de ejercicios, ofreciendo retroalimentación y monitoreando el avance del estudiante.

En relación con la seguridad del Backend, se han establecido roles y restricciones de acceso coherentes con las reglas de negocio previamente determinadas. Aunque la arquitectura actual no posee herramientas concretas orientadas a la escalabilidad, está concebida para ser resistente y estable.

Es esencial destacar que, la estructura general del sistema sigue el patrón Modelo-Vista-Controlador (MVC). La Vista reside en el Frontend, y el Controlador y Modelo en el Backend 6.1. Este patrón facilita la distinción entre la lógica de la interfaz de usuario y las operaciones, simplificando su mantenimiento.

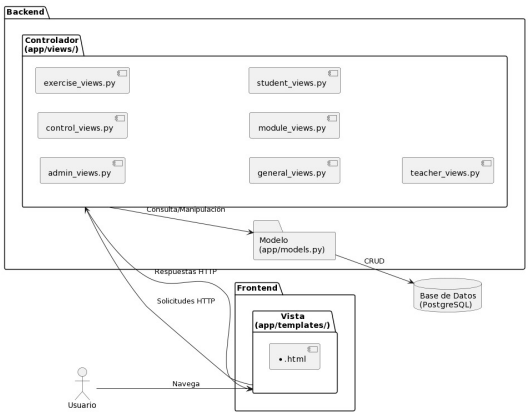

Figura 6.1: Arquitectura de sistema

6.3. Diseño de Base de Datos

La arquitectura de nuestra base de datos es el resultado de un exhaustivo análisis y diseño. Su estructura no solo atiende a las necesidades actuales, sino que también se anticipa a los retos futuros. A continuación, detallamos algunos de los elementos clave y las consideraciones adoptadas en su construcción:

1. **Tablas principales y su significado**: Las tablas como *users*, *modules*, *exercises*, *questions* y *theory* son la columna vertebral de la base de datos. En ellas, se almacena la información medular que alimenta las funcionalidades centrales del sistema. Estas tablas, además de contener datos primordiales, están interconectadas a través de relaciones específicas. Garantizando, así, la coherencia y la integridad de la información en todo momento.

2. **La importancia de los índices**: Los índices son herramientas fundamentales para acelerar las consultas y garantizar un rendimiento óptimo. Dentro de la base de datos, su uso es estratégico. Existen índices únicos, como el *users_email_key*, que aseguran la singularidad de ciertos registros. Por ejemplo, en el caso del correo electrónico, se garantiza que no existan duplicados. Los índices compuestos, como *student_modules_student_id_module_id_key*, son esenciales para optimizar las consultas que abarcan múltiples campos. Las tablas intermedias, que representan relaciones entre otras, también hacen uso de índices. En este caso, *exerciserequirements* y *theoryrequirements*. Estos índices, principalmente compuestos, agilizan y refuerzan las consultas relacionales.

3. **La normalización como estándar**: Uno de los principios fundamentales que se ha seguido es la normalización. Al estructurar la información de esta manera, se minimiza la redundancia y maximiza la eficiencia.

4. **Relaciones bien definidas**: La claridad en las relaciones entre tablas es esencial. Gracias a las claves primarias y foráneas, se ha conseguido una red de relaciones que no solo facilita la integración y consulta de datos sino que también refuerza la integridad de los mismos.

5. **Mirando hacia el futuro**: Más allá de las necesidades actuales, el diseño busca ser resiliente y adaptable. Pretendiendo que cualquier adición futura, ya sea en funcionalidades o módulos, se integre sin grandes complicaciones.

En resumen, la base de datos del sistema es robusta, detallada y diseñada para crecer. Teniendo como aspiración que, más allá de ser un simple almacén de datos, sea una herramienta eficiente que potencie la experiencia del usuario y facilite el trabajo de la ITS.

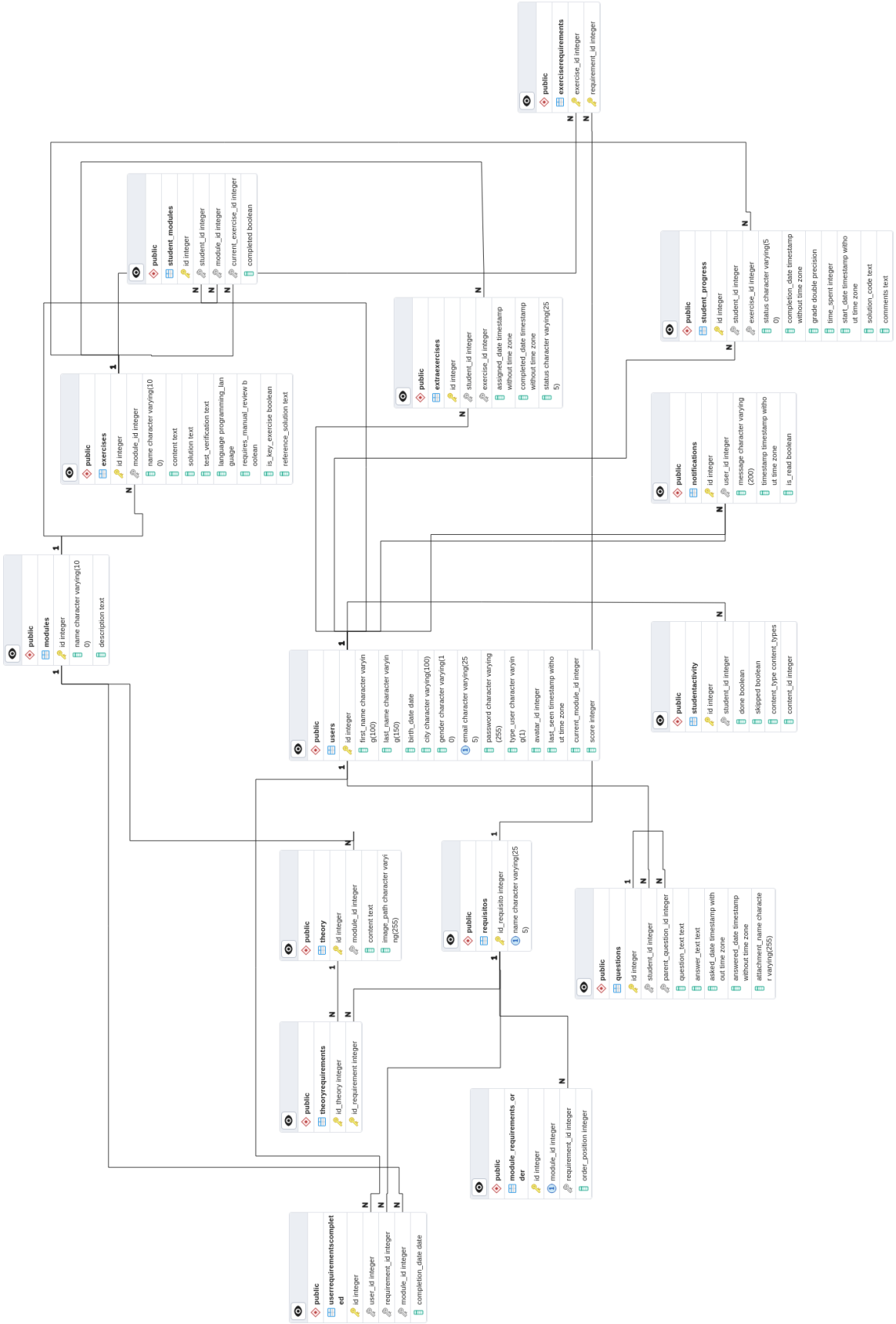

Figura 6.2: Diagrama del modelo de datos

6.4. Interfaz de Usuario

6.4.1. *Mockups* de la página web

Este apartado incluye una serie de *mockups* que representan las diferentes interfaces de usuario del sistema. Los *mockups* han sido diseñados utilizando la herramienta en línea *wireframe.cc* [25]. Esta herramienta permite la creación rápida y eficiente de prototipos de interfaces, facilitando la visualización y el diseño preliminar de las interacciones del usuario que se muestran a continuación.

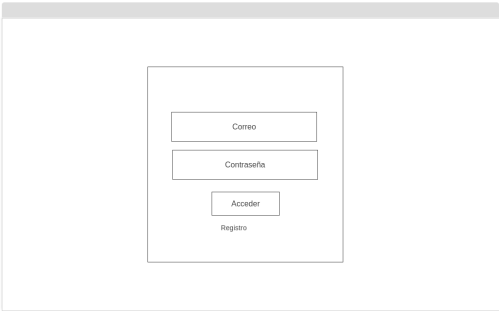

Figura 6.3: Inicio de Sesión para cualquier usuario.

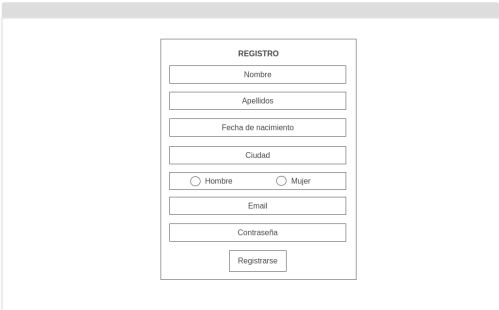

Figura 6.4: Registro para cualquier usuario.

Figura 6.5: Página Principal de la interfaz del estudiante.

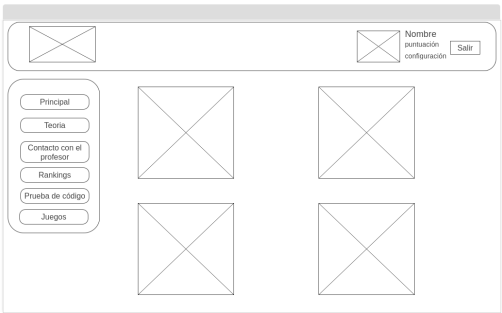

Figura 6.6: Apartado de Juegos de la interfaz del alumno.

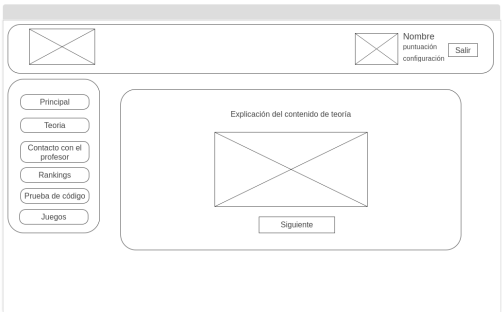

Figura 6.7: Visualización de la teoría para el estudiante.

Figura 6.8: Apartado de contacto con el profesor.

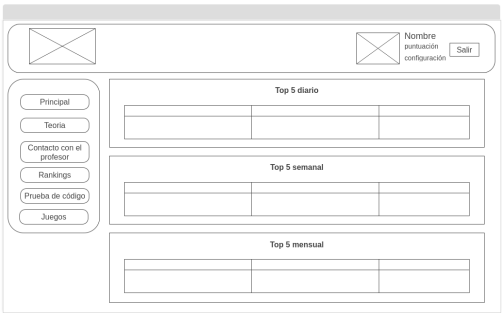

Figura 6.9: Apartado de Ranking de la interfaz del usuario.

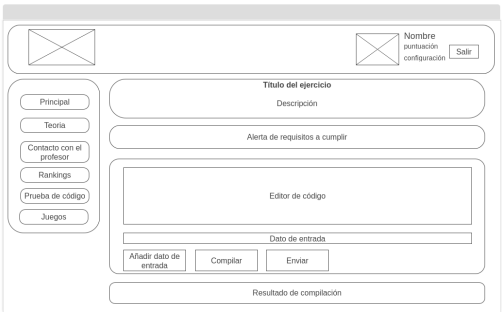

Figura 6.10: Visualización de los ejercicios para el usuario.

Figura 6.11: Apartado donde los estudiantes pueden revisar conceptos teóricos.

Figura 6.12: Apartado de configuración del estudiante.

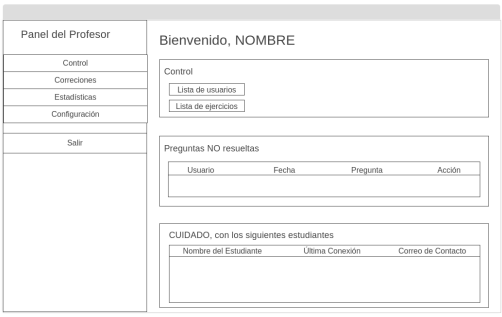

Figura 6.13: Dashboard principal del profesor.

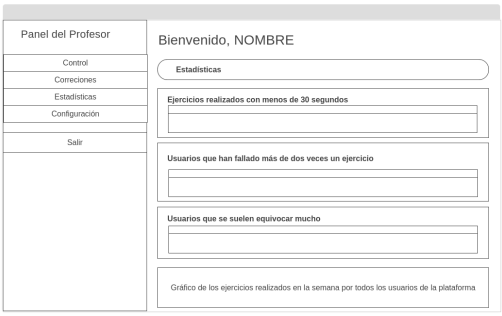

Figura 6.14: Pantalla de Estadísticas del profesor.

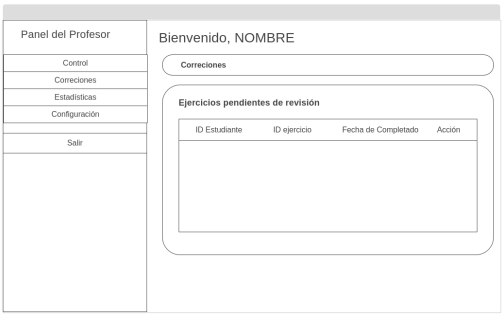

Figura 6.15: Pantalla con la lista de corrección de ejercicios.

Figura 6.16: Panel para facilitar la corrección los ejercicios para el profesor.

Figura 6.17: Apartado de Configuración del profesor.

Figura 6.18: Visión general del panel administrativo.

6.4.2. Interacción Usuario-Sistema

El sistema está diseñado para una comunicación efectiva y amigable con el usuario. La fluidez en la navegación y la clara representación de opciones forman una experiencia de usuario destacable.

1. **Inicio de Sesión**: Al entrar, la primera pantalla es la de inicio de sesión. Aquí, el usuario introduce su nombre y contraseña. Si las credenciales son correctas, se redirige a la página principal. De lo contrario, un breve mensaje de error informa del problema.

2. **Registro**: Para nuevos usuarios, hay una pantalla especial donde se pide la información básica.

3. **Página Principal**: Una vez dentro, la página principal se despliega. Ofrece múltiples funciones: desde juegos hasta rankings. Iconos y etiquetas guían al usuario, haciendo la navegación intuitiva.

4. **Juegos, Teoría y Ejercicios**: Estas secciones son el corazón educativo del sistema. Los juegos son la motivación del estudiante. La teoría ofrece contenido educativo valioso y los ejercicios proponen retos prácticos que evalúan el aprendizaje del usuario.

5. **Contacto con el profesor y Ranking**: Dos secciones cruciales. En 'Contacto con el profesor', el usuario puede realizar consultas y ver sus anteriores dudas conjuntamente con los comentarios de algunos ejercicios. 'Ranking' motiva a los usuarios mostrando una lista de los más destacados.

6. **Interfaz del Profesor**: No solo los estudiantes tienen espacio en el sistema. Los profesores tienen su propia interfaz. Pueden observar estadísticas, corregir tareas y más. También tienen acceso a un panel administrativo.

7. **Notificaciones**: Finalmente, las notificaciones juegan un papel vital. Informan al usuario sobre correcciones, nuevos contenidos o interacciones. Esta comunicación directa asegura que el usuario esté siempre al tanto de los eventos relevantes.

El **Panel del Administrador** es una herramienta esencial para la gestión de la base de datos. Concretamente, ofrece una interfaz gráfica que evita la necesidad de interactuar a través de la terminal. Este enfoque agiliza las tareas cotidianas, y reduce la posibilidad de errores humanos que podrían surgir al gestionar datos de forma manual. Es importante mencionar que solo hemos mostrado un contexto general de la interfaz dentro de los mockups presentados. Estas representaciones cubren las funciones básicas, por ello, incluir más detalles sería redundante, ya que las operaciones fundamentales están claramente ilustradas.

En resumen, el sistema busca una interacción amena y productiva proporcionando las herramientas que necesitan los diferentes usuarios, presentadas de manera clara y accesible.

6.5. Implementación de Algoritmos

6.5.1. Corrección y evaluación de calidad del código

El verdadero reto comienza tras la entrega de un ejercicio por parte del estudiante. La lógica de corrección evalúa el código en términos de precisión y calidad. Si un código no alcanza un estándar mínimo, el sistema propone ejercicios adicionales, reforzando así las áreas de mejora del estudiante 6.19. La justificación detrás de esta lógica se centra en la premisa de que la repetición y el refuerzo son esenciales para consolidar el aprendizaje. Además, al establecer

estándares de calidad, se fomentan las buenas prácticas de programación desde las etapas iniciales de formación.

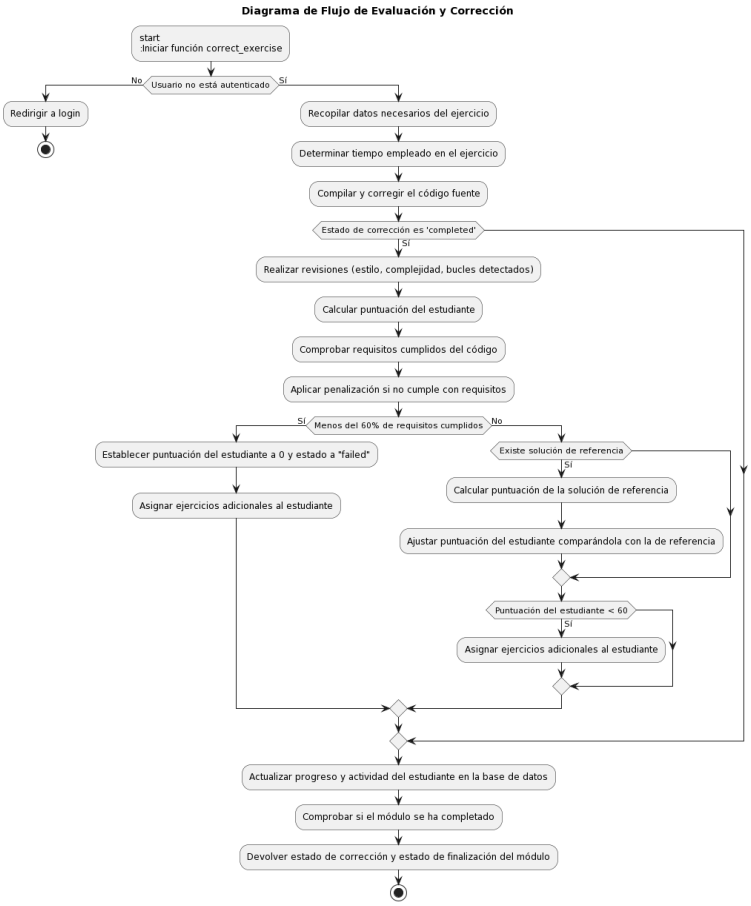

Figura 6.19: Diagrama de flujo para la selección de ejercicios

6.5.2. Selección y evaluación de ejercicios

El sistema prioriza la progresión estructurada del estudiante. Al ingresar, la plataforma identifica ejercicios en curso o determina el próximo paso en función de los requisitos del módulo. Esta estructura garantiza que, antes de enfrentarse a cualquier tarea, los conceptos teóricos relevantes sean presentados al estudiante 6.20. Esta estrategia se fundamenta en la pedagogía moderna, que sugiere que la teoría y la práctica deben ir de la mano para un aprendizaje óptimo.

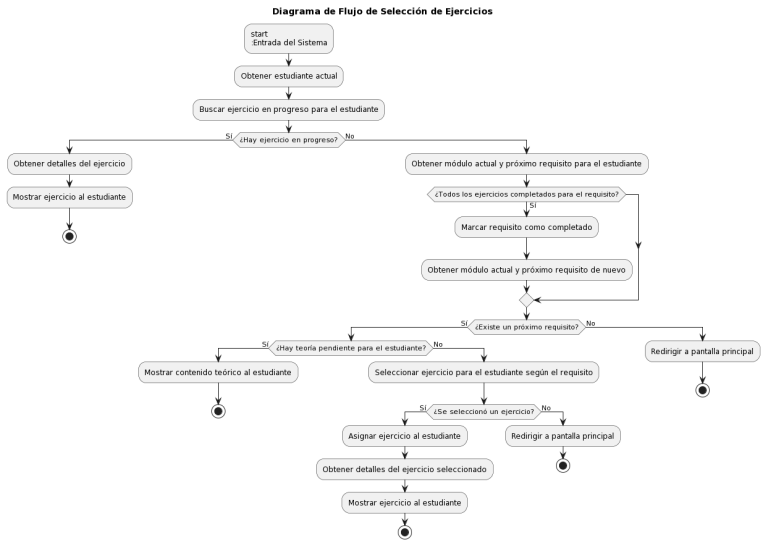

Figura 6.20: Diagrama de flujo para la selección de ejercicios

6.5.3. Cálculo de la puntuación

El sistema utiliza la función `calculate_score` para calcular la puntuación de un ejercicio. Esta función combina múltiples criterios que inciden en la calidad y eficacia del código del estudiante.

- **Problemas de Estilo**: Detectados mediante un análisis estático. Cada inconveniente identificado resta 5 puntos de los 100 iniciales.
- **Complejidad Ciclomática**: Un valor elevado (superior a 10) en la complejidad ciclomática suele indicar que el código puede ser difícil de mantener o probar. Por ello, se aplica una penalización significativa de 20 puntos. Esta cantidad ha sido establecida para subrayar la importancia de escribir código sencillo y fácilmente comprensible.
- **Bucles Detectados**: La presencia de bucles no es necesariamente un problema, pero puede llevar a un código menos eficiente o más complejo si no se usan adecuadamente. Se resta una cantidad menor, 15 puntos, con tal de incentivar el uso cuidadoso de estas estructuras.

La fórmula que resume todo el proceso es la siguiente:

$$\text{máx}\,(0, 100 - 5 \times \text{Problemas de Estilo} - 20 \times I_{\text{complejidad } >10} - 15 \times I_{\text{bucles detectados}}) \tag{6.1}$$

Cabe destacar que $I_{\text{condición}}$ actúa como una función indicadora. Toma el valor de 1 si la condición se cumple y 0 en caso contrario.

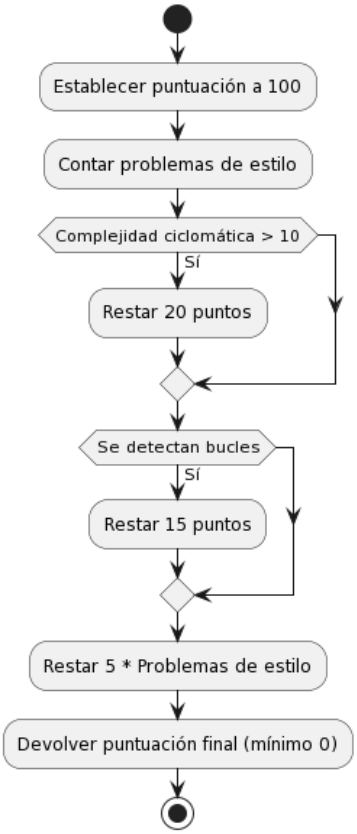

Figura 6.21: Diagrama de flujo que ilustra cómo se calcula la puntuación

Finalmente, la función tiene un control para que la puntuación no sea negativa, con tal de evitar la desmotivación del estudiante. Devuelve el máximo entre el valor calculado y cero.

6.5.4. Comprobaciones del código

Para evaluar el cumplimiento de ciertos requisitos educativos en el código, se emplea la técnica de Análisis de Árbol de Sintaxis Abstracta (AST) [20]. En este contexto, se utiliza una clase especializada denominada `RequirementVisitor`, que hereda de la clase base `ast.NodeVisitor`.

La clase `RequirementVisitor` es responsable de recorrer el AST generado a partir del código fuente. En su recorrido, identifica diferentes nodos que corresponden a constructos programáticos específicos: operadores básicos, estructuras de control, funciones, clases, entre otros.

Una vez que se encuentra un nodo de interés, la clase cambia el estado de una bandera correspondiente en un diccionario llamado *requirements_found* a True. Este diccionario actúa como un registro para los diferentes requisitos que el código cumple o no cumple.

Este mecanismo proporciona una manera eficiente y precisa de evaluar qué conceptos de programación se han aplicado en un determinado fragmento de código, lo que a su vez permite una evaluación pedagógica más informada.

Concretamente, dentro del constructor de la clase, se inicializa el diccionario con tal de tener un registro de los distintos elementos sintácticos y estructurales detectados en el código. Con el método `visit_BinOp`, se comprueba la presencia de operadores básicos como suma, resta, multiplicación y división. Usando `visit_BoolOp` se identifica el uso de operadores lógicos como AND y OR. El uso de las declaraciones `if-else` se detecta con el método `visit_If`. Los métodos `visit_While` y `visit_For` se encargan de identificar bucles `while` y `for`, respectivamente. Se detecta la definición de funciones mediante `visit_FunctionDef`. El método `visit_ClassDef` no solo detecta la definición de clases sino también la herencia mediante el atributo `bases`. Finalmente, `visit_List` y `visit_Dict` identifican el uso de listas y diccionarios, respectivamente.

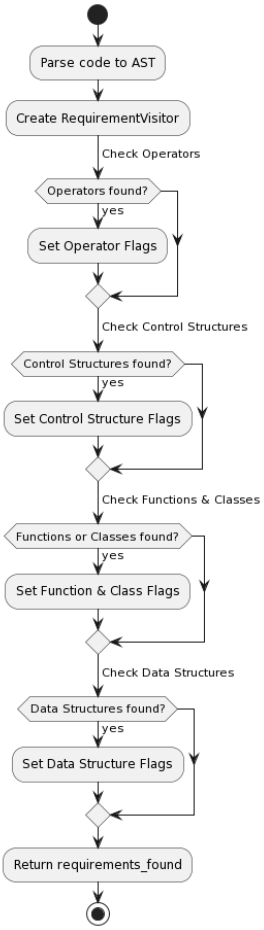

Figura 6.22: Diagrama de flujo que ilustra cómo se calcula la puntuación

6.5.5. Cálculo del número extra de ejercicios

El principal objetivo de la función `determine_number_of_extra_exercises` es adaptar de manera dinámica la carga de ejercicios asignados a un estudiante en función de su desempeño reciente, ofreciendo una experiencia de aprendizaje más personalizada.

Parámetros de Entrada

La función acepta los siguientes argumentos:

- **student_id**: Este identificador único del estudiante se utiliza para acceder a sus registros y recuperar su historial de rendimiento.
- **recent_num = 4**: Este parámetro establece que se tendrán en cuenta los últimos 4 ejercicios completados para evaluar la tasa de fracaso del estudiante. El número 4 se ha seleccionado como un equilibrio entre tener una muestra lo suficientemente grande para ser significativa, pero no tan grande como para ser desactualizada.
- **max_extra_exercises = 5**: Este es el número máximo de ejercicios adicionales que se pueden asignar. Limitar esto a 5 ayuda a prevenir una carga de trabajo abrumadora y repetitiva para el estudiante.
- **failure_rate_threshold = 0.35**: Se asignarán ejercicios adicionales si la tasa de fracaso supera este umbral. El valor de 0.35 se establece para permitir cierto margen de error pero todavía requiere una intervención si el rendimiento del estudiante es consistentemente bajo.

Flujo de Trabajo y Razonamiento

El flujo de trabajo de la función se puede desglosar en las siguientes etapas:

1. **Consulta de Requisitos**: La función primero identifica el último "requisito" completado por el estudiante para determinar desde dónde empezar.
2. **Búsqueda de Ejercicios Relacionados**: Se recuperan los IDs de los ejercicios asociados con el requisito actual para tener un conjunto de datos sobre el cual operar.
3. **Recolección y Análisis de Datos**: A continuación, se evalúa el rendimiento del estudiante en estos ejercicios, especialmente en términos de fracasos y éxitos, para calcular una tasa de fracaso.

4. **Normalización del Tiempo Empleado**: Los tiempos empleados en los ejercicios se normalizan para darles una escala uniforme, lo que permite una mejor comparación.
5. **Ponderación y Cálculo Final**: Los dos factores (tasa de fracaso y tiempo normalizado) se combinan en una suma ponderada para determinar el número final de ejercicios extra.

La fórmula utilizada para calcular el número de ejercicios extra es la siguiente:

$$N = \text{máx}\left(0, \text{round}\left(w_f \times f + w_t \times t\right) \times M\right) \tag{6.2}$$

Donde:

- $w_f = 0.7$ y $w_t = 0.3$ son los pesos asignados a la tasa de fracaso y al tiempo, respectivamente. Estos pesos reflejan la importancia relativa de cada factor; en este caso, la tasa de fracaso se considera más crítica.
- f es la tasa de fracaso actual del estudiante.
- t es el tiempo normalizado del último ejercicio.
- M es el número máximo de ejercicios extra permitidos.

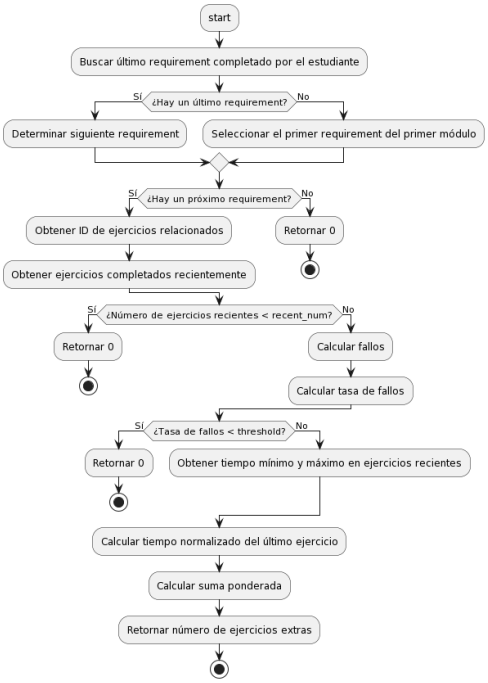

Figura 6.23: Diagrama de flujo que ilustra el cálculo del número extra de ejercicios

6.6. Implementación de Módulos de Aprendizaje Adaptativo

El núcleo de nuestro sistema educativo se centra en un modelo de aprendizaje adaptativo, diseñado para responder de manera dinámica al progreso y las necesidades del estudiante. La idea es no solo adaptar el tipo de material de aprendizaje presentado, sino también ajustar la cantidad y complejidad de los ejercicios asignados, todo en tiempo real.

La adaptación del sistema se logra mediante una combinación de técnicas de análisis de datos. En primer lugar, se recopilan métricas clave del rendimiento del estudiante, como las tasas de éxito y fracaso en ejercicios, el tiempo empleado. Estos datos se almacenan y analizan para determinar patrones y tendencias que luego son utilizadas para tomar decisiones adaptativas.

En el nivel más básico, el sistema cuenta con un conjunto predefinido de requisitos de aprendizaje, que son objetivos educativos específicos que el estudiante debe alcanzar. A medida que el estudiante avanza, el sistema evalúa continuamente su rendimiento y ajusta los requisitos de aprendizaje en función de este. Por ejemplo, si un estudiante supera con facilidad los ejercicios iniciales en un tema, el sistema puede decidir si avanzarle a un nuevo tema más rápidamente.

Además, en situaciones donde un estudiante demuestra dificultades persistentes, como una tasa de fracaso elevada en ejercicios recientes, el sistema asigna automáticamente ejercicios adicionales de refuerzo con tal ayudar al estudiante a mejorar en las áreas problemáticas, como se detalla en la sección anterior sobre el cálculo del número extra de ejercicios.

Un aspecto crítico que distingue nuestra implementación es la adaptabilidad basada en una solución de referencia proporcionada por el profesor. Este enfoque mitiga cualquier rigidez en el proceso de evaluación y garantiza que la retroalimentación sea contextual y ajustada.

El ajuste de la puntuación del estudiante se realiza mediante las siguientes fórmulas para calcular el fitness y la puntuación respectivamente:

$$\text{máx}\left(0, 1 - \frac{|\text{teacher_score} - \text{student_score}|}{100}\right) \tag{6.3}$$

$$\text{student_score} \times = \text{fitness} \tag{6.4}$$

La variable `fitness` calcula la discrepancia entre la puntuación del estudiante y la del profesor. Este valor luego multiplica la puntuación original del estudiante para ajustarla.

En resumen, la implementación de módulos de aprendizaje adaptativo en nuestro sistema se lleva a cabo a través de un análisis cuidadoso de los datos del rendimiento del estudiante, seguido de la toma de decisiones algorítmicas para personalizar la experiencia de aprendizaje. Esta aproximación permite

no solo una mayor eficiencia en el proceso educativo, sino también una experiencia más personalizada y, por ende, más atractiva para el estudiante.

Capítulo 7

Pruebas y Resultados

Con el fin de evaluar exhaustivamente el sistema implementado, se realizaron pruebas con un pequeño pero significativo grupo de niños, compuesto por 3 participantes de distintas edades. Concretamente de 8, 12 y 15 años. Cada uno de ellos interactuó con el sistema de manera individual, en sesiones separadas. De esta manera, se obtuvo una evaluación más personalizada, detallada y precisa de la eficacia del sistema.

Los indicadores cuantitativos mostraron resultados bastante alentadores. Los jóvenes usuarios lograron completar los primeros ejercicios con éxito. Además, el tiempo que pasaron interactuando con la plataforma se mantuvo en un rango óptimo. Por lo tanto, se puede decir que el sistema fue eficaz en adaptarse a dificultad. Sin embargo, se observó que el tiempo óptimo de interacción con el sistema debería ajustarse según la edad del usuario, dado que las capacidades de atención y ritmo de lectura varían considerablemente en función de la edad.

En términos de usabilidad, las respuestas fueron positivas. La plataforma fue calificada como fácil de usar y práctica, lo cual indica que el diseño de la interfaz es adecuado. La claridad en las instrucciones también fue un punto destacado aunque a veces desconcertante para los más pequeños.

Entre las sugerencias aportadas, la más recurrente fue la adición de elementos gamificados con tal de incrementar la motivación y el compromiso. Este tipo de retroalimentación fue especialmente valiosa y se consideró como una mejora importante a realizar en el sistema. Además, la metodología de pruebas individuales permitió recoger opiniones y sugerencias sin la influencia del grupo.

Para concluir, las pruebas iniciales sugirieron que el sistema es eficiente y efectivo en su misión de ofrecer una experiencia de aprendizaje adaptativo. Los comentarios y observaciones de los jóvenes participantes ofrecen un conjunto invaluable de datos que han servido para optimizar y enriquecer el sistema en futuras versiones.

Capítulo 8

Conclusiones y Líneas de Trabajo Futuro

Apéndice A

Uso de la Plataforma

Glosario

ITS: Sistema Inteligente de Tutorización (Intelligent Tutoring System). Es un sistema que utiliza tecnologías de inteligencia artificial para proporcionar enseñanza personalizada y apoyo educativo a los estudiantes, adaptándose a las necesidades individuales de aprendizaje.

Bibliografía

[1] Plantulm. `https://plantuml.com/es/`, 2023. Accedido el: 29 de Octubre, 2023.

[2] Bernat Foj Capell. Codelearn. `https://www.codelearn.es`, 2013. Accedido el: 24 de Octubre, 2023.

[3] Tyne Crow, Andrew Luxton-Reilly, and Burkhard Wünsche. Intelligent tutoring systems for programming education: a systematic review. pages 53–62, 01 2018.

[4] Servicio de Innovación Educativa. Aprendizaje basado en problemas. *Guías rápidas sobre nuevas metodologías, Disponible en http://innovacioneducativa. upm. es/guias/Aprendizaje_ basado_ en_ problemas. pdf*, 2008.

[5] Robert B.K. Dewar and Edmond Schonberg. Computer science education: Where are the software engineers of tomorrow? *Open Forum*, 2001.

[6] DragonBox. The dragonbox method. `https://dragonbox.com/`, 2022. Accedido el: 25 de Octubre, 2023.

[7] Discovery Education. Dreambox learning. `http://www.dreambox.com/`, 2022. Accedido el: 24 de Octubre, 2023.

[8] Douglas Frye and Elliot Soloway. Interface design: A neglected issue in educational software. In *Proceedings of the SIGCHI/GI Conference on Human Factors in Computing Systems and Graphics Interface*, CHI '87, pages 93–97. Association for Computing Machinery, 1986.

[9] Inc. GlassLab y Take-Two Interactive Software. Civilization edu. `https://civilization.com/es-ES/`, 2016. Accedido el: 25 de Octubre, 2023.

[10] Google. Google it support professional certificate. `https://grow.google/it-cert/`, 2023. Accedido el: 25 de Octubre, 2023.

[11] IIS International. *Adaptative Learning in real world contexts*, Zagreb, 2016. 13ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[12] IIS International. *Artifical Intelligence and Beyond: from Alpha to Omega*, Athens (Online Conference), 2020. 16ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[13] IIS International. *Intelligent Tutoring Sytems in an Online world*, Athens (Online Conference), 2021. 17ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[14] Andrew Ng Daphne Koller. Coursera for campus. `https://www.coursera.org/campus`, 2011. Accedido el: 25 de Octubre, 2023.

[15] J. A. Large and Jamshid Beheshti. Interface design, web portals, and children. *Library Trends*, 54(2):318–342, 2005.

[16] Carnegie Learning. Fast forword. `https://www.scilearn.com/#`, 2021. Accedido el: 24 de Octubre, 2023.

[17] Lockheed Martin. F-35 lightning ii training system. `https://www.lockheedmartin.com/en-us/products/f-35-lightning-ii-training-systems.html`, 2021. Accedido el: 25 de Octubre, 2023.

[18] Medtronic. Touch surgery simulator. `https://www.touchsurgery.com/simulations`, 2019. Accedido el: 25 de Octubre, 2023.

[19] Arnold Pears, Stephen Seidman, Lauri Malmi, Linda Mannila, Elizabeth Adams, Jens Bennedsen, Marie Devlin, and James Paterson. A survey of literature on the teaching of introductory programming. *ACM SIGCSE Bulletin*, 39(4):204–223, 2007.

[20] Python. Librería ast — Árboles de sintaxis abstracta. `https://docs.python.org/es/3/library/ast.html`, 2001-2023. Accedido el: 30 de Octubre, 2023.

[21] Anthony Robins, Janet Rountree, and Nathan Rountree. Learning and teaching programming: A review and discussion. *Computer Science Education*, 13(2):137–172, 2003.

[22] Seguridad Social. Cotización y recaudación de trabajadores para el 2021. `https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537`, 2023. Accedido el: 26 de Octubre, 2023.

[23] Ugur Tevfik Kaplancali and Zafer Demirkol. Teaching coding to children: A methodological approach. *International Journal of Elementary Education*, 6(4):32–37, 2017.

[24] Noelia Carrasco Vilar. Repositorio del proyecto. `https://github.com/noocaaa/TFG_WEB/`, 2023. Accedido el: 30 de Octubre, 2023.

[25] Wireframe.cc. Wireframe. http://www.wireframe.cc/, 2023. Accedido el: 30 de Octubre, 2023.

[26] MIT y Harvard. edx. https://www.edx.org, 2012. Accedido el: 25 de Octubre, 2023.